

Embodied Vision

Physical Modeling: Time Integration, Mass-Spring System
and Finite Element Method

Tsung-Wei Ke

Spring 2026

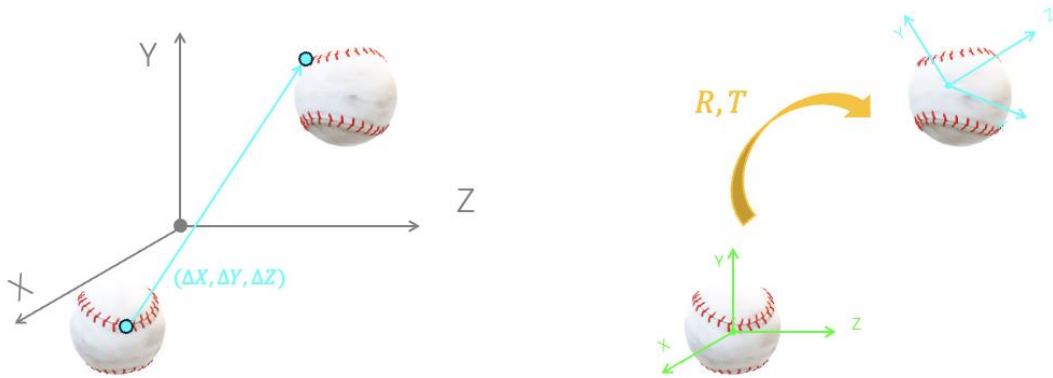


Disclaimer

- This lecture borrows contents heavily from
 - [Physics-based Animation](#) by David I.W Levin at University of Toronto
 - [Physics-based Animation of Solids and Fluids](#) by Minchen Li at CMU
- This is an introductory lecture to physical modeling. Check the advanced courses in physic-based simulation for more details.

Recap

Rigid-body Motion



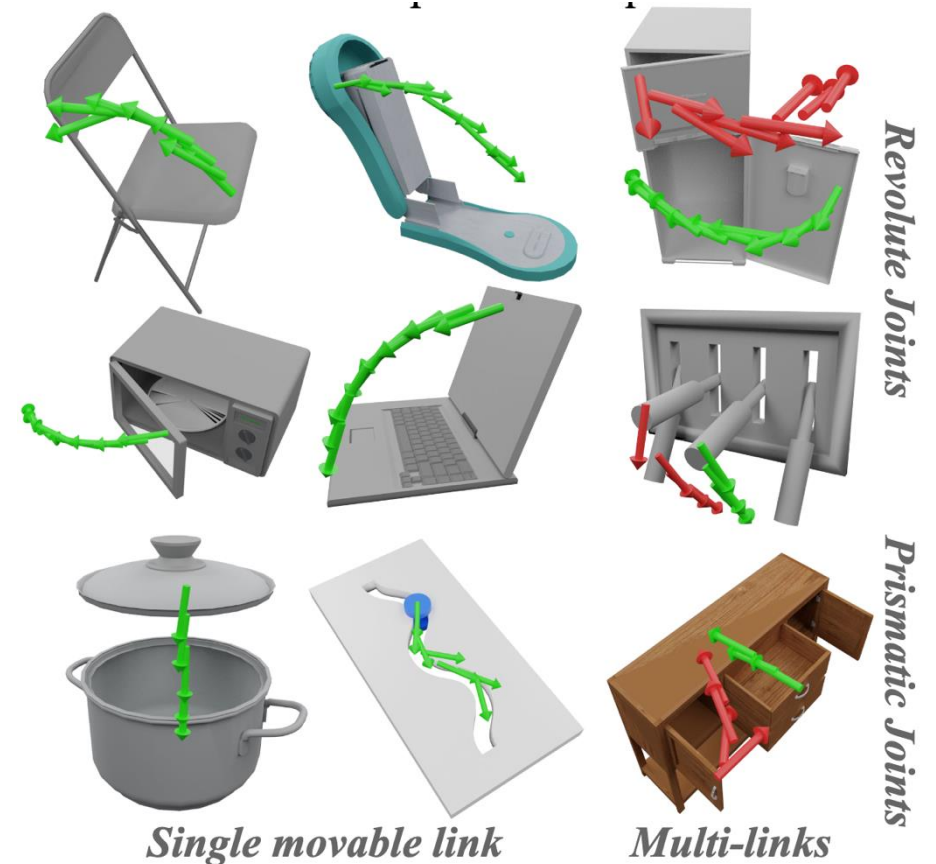
Rigid-body physic law:

$$\frac{d}{dt}\mathbf{q}(t) = \frac{d}{dt} \begin{bmatrix} \mathbf{t}(t) \\ \mathbf{R}(t) \\ \boldsymbol{\nu}(t) \\ \boldsymbol{\omega}(t) \end{bmatrix} = \begin{bmatrix} \mathbf{v}(t) \\ \boldsymbol{\omega}(t) \times \mathbf{R}(t) \\ \frac{\mathbf{F}(t)}{M} \\ \mathbf{I}(t)^{-1}(\boldsymbol{\tau} - \boldsymbol{\omega}(t) \times \mathbf{I}(t)\boldsymbol{\omega}(t)) \end{bmatrix}$$

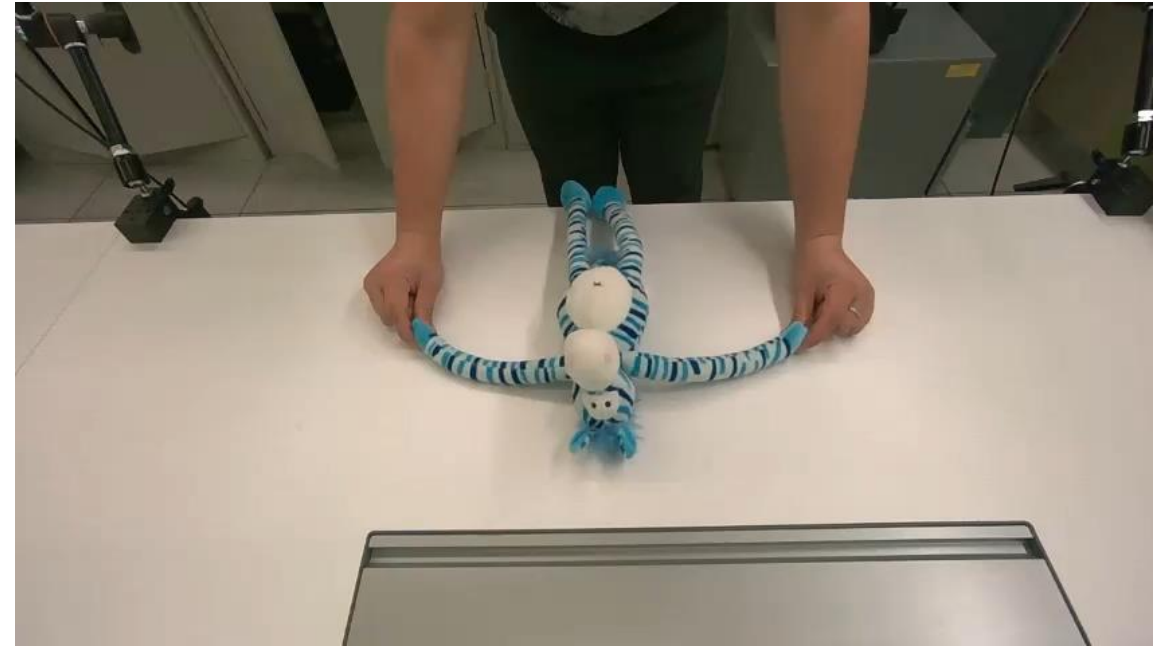
Simulation:

$$\mathbf{q}(t) = \mathbf{q}(0) + \int_0^t \frac{d}{dt}\mathbf{q}(t)dt = \mathbf{q}(0) + \sum_{i=1}^T \frac{d}{dt}\mathbf{q}(t)|_{t=t_i} \Delta t$$

Articulated-object Motion



However, Most Objects are not Rigid Bodies...



However, Most Objects are not Rigid Bodies...



Cloth



Rope



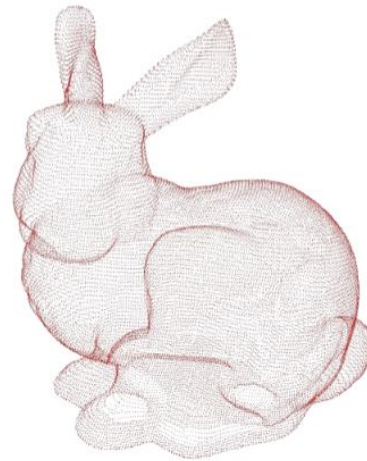
Fluid



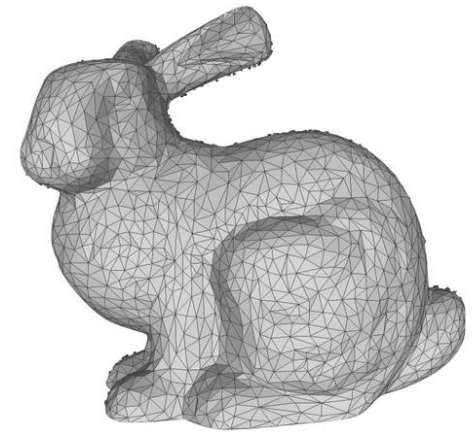
Soft body

How to Physically Model Non-Rigid-Body Motion?

1. How to represent 3D model of non-rigid bodies?



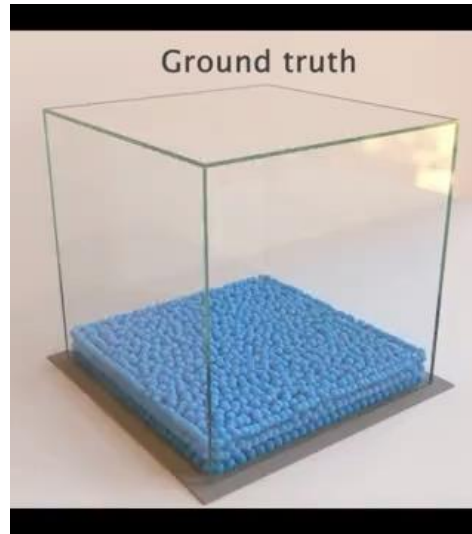
point



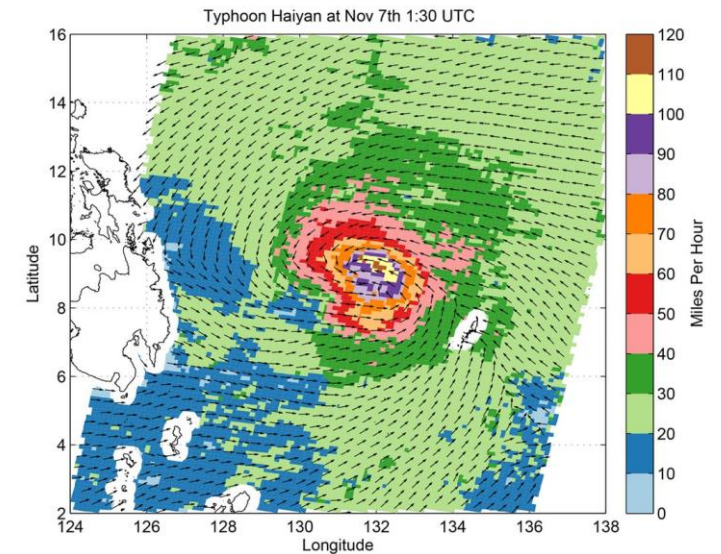
mesh

How to Physically Model Non-Rigid-Body Motion?

1. How to represent 3D model of non-rigid bodies?
2. How to represent non-rigid-body motion?



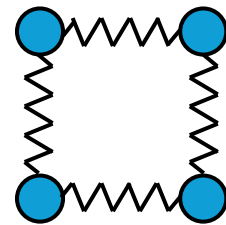
Particle motion



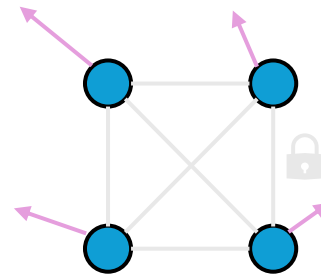
Motion field

How to Physically Model Non-Rigid-Body Motion?

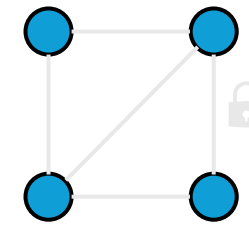
1. How to represent 3D model of non-rigid bodies?
 2. How to represent non-rigid-body motion?
 3. How to physically model non-rigid-body motion?
- Note that point motions are correlated



Mass-spring
system

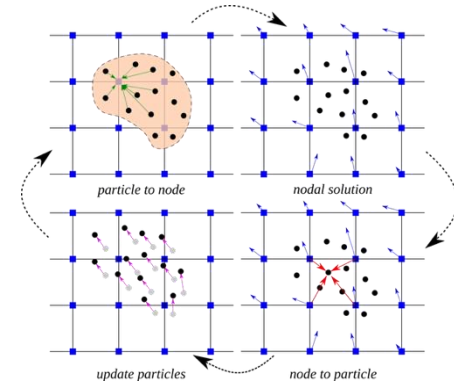


Position Based
Dynamics



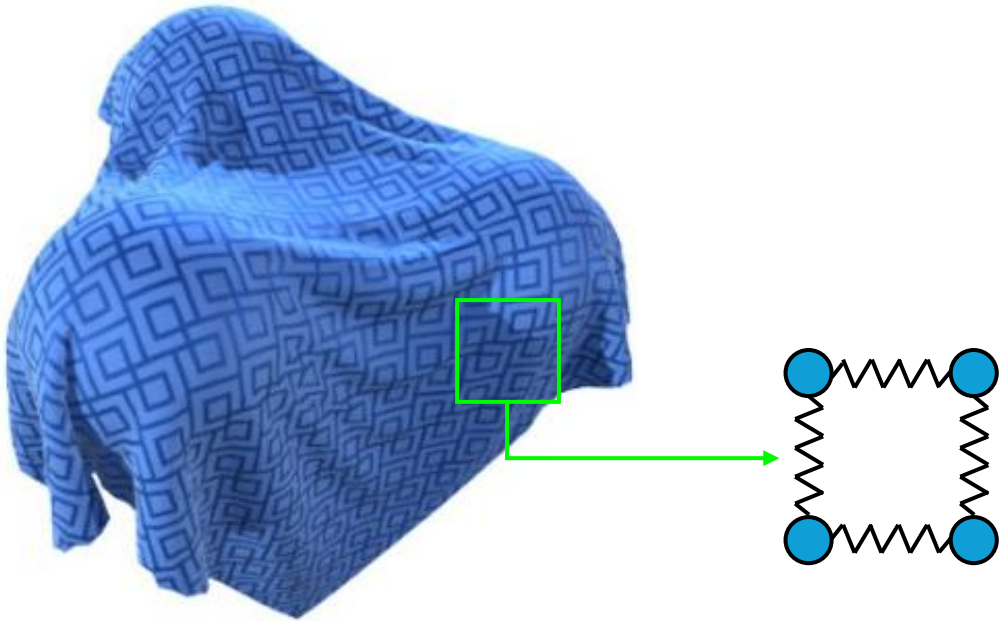
$$\min_x E(x, x^t, v^t) \quad \text{s.t.} \quad g(x) \geq 0.$$

Incremental Potential
Solver



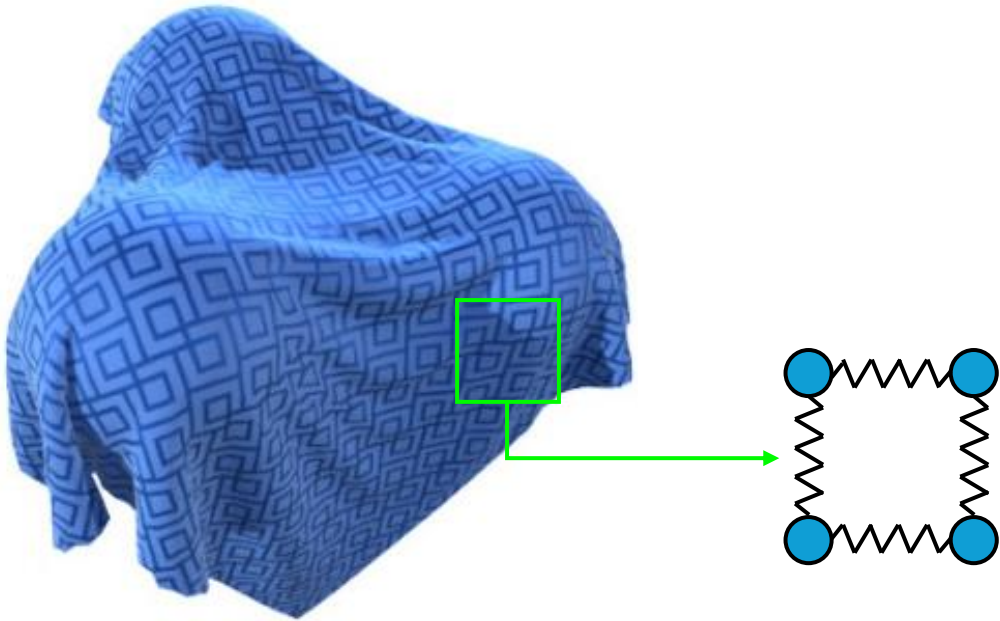
Material Point
Method

Mass-Spring System



- **Idea:** Represent a deformable object as a set of mass nodes $\mathcal{V}$ connected by spring-based edges $\mathcal{E}$, forming a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$
- The connectivity pattern is pre-defined
- The dynamic formulation of a mass-spring system:
 - Calculate the total force on each node
 - Update the velocity and position of each node

Mass-Spring System



- The dynamic formulation of a mass-spring system:
 - The total force on node i :

$$F_i = \sum_{i,j \in \mathcal{E}} F_{i,j}^{spring} + F_{i,j}^{dashpot} + F_i^{ext}$$

- Spring force describes the bounding force between nodes:

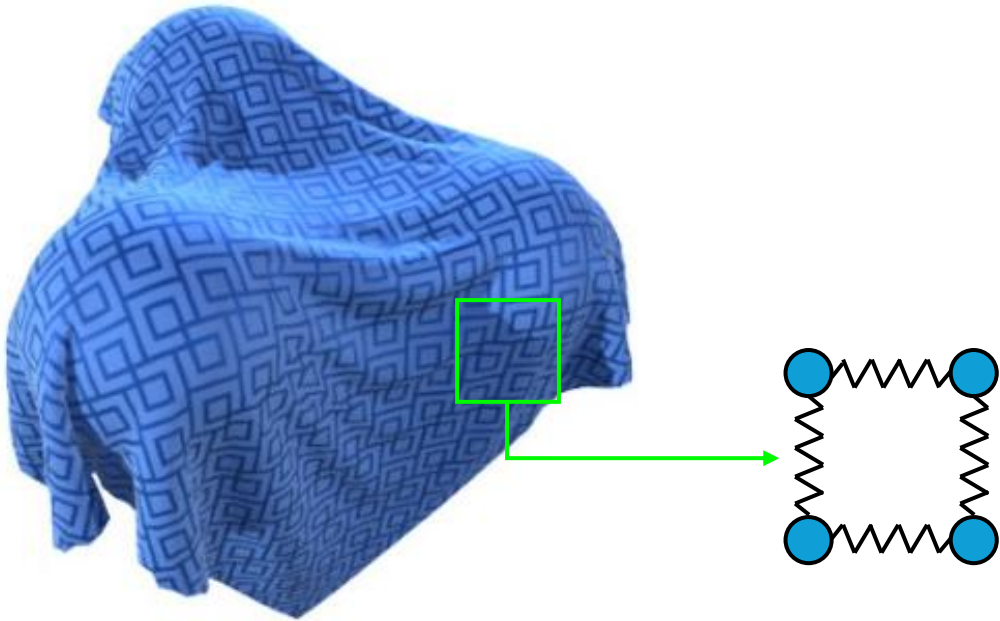
$$F_{i,j}^{spring} = k_{i,j} (\|x_j - x_i\| - l_{i,j}) \frac{x_j - x_i}{\|x_j - x_i\|}$$

- The dashpot dissipates energy, preventing oscillations:

$$F_{i,j}^{dashpot} = -\gamma (\|v_i - v_j\|)$$

- External forces accounts for collision, gravity and user interactions

Mass-Spring System



- The dynamic formulation of a mass-spring system:

➤ The total force on node i :

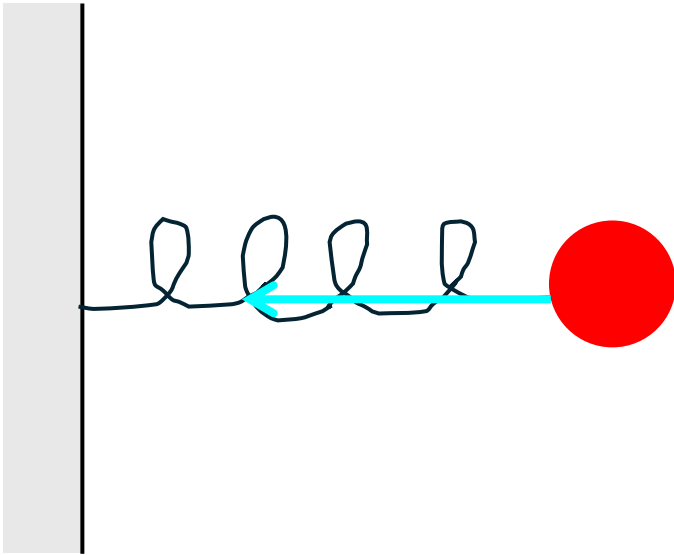
$$F_i = \sum_{i,j \in \mathcal{E}} F_{i,j}^{spring} + F_{i,j}^{dashpot} + F_i^{ext}$$

➤ The updated velocity and position of node i :

$$\begin{aligned} v_i^{t+1} &= \delta \left(v_i^t + \Delta t \frac{F_i}{m_i} \right) \\ x_i^{t+1} &= x_i^t + \Delta t v_i^{t+1} \end{aligned}$$

How do we derive this update rule?
Is this update rule optimal?

Let's Consider a Simple Case



- The total force on node i :

$$F_i = \sum_{i,j \in \mathcal{E}} F_{i,j}^{spring} + \cancel{F_{i,j}^{dashpot}} + \cancel{F_i^{ext}}$$

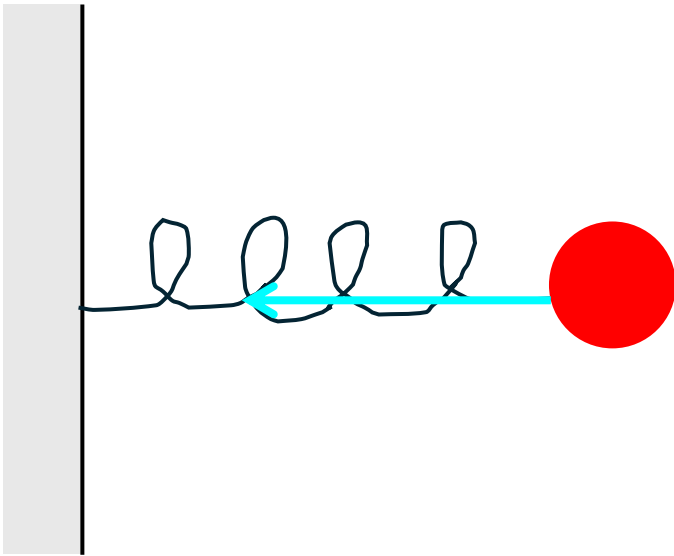
- Another end is attached to the wall: $x_j = 0$
- The rest length of the spring is zero: $l_{i,j} = 0$
- Spring force:

$$F_i^{spring} = -kx_i(t)$$

- Newton's second law:

$$F_i = m\ddot{x}_i(t) \Rightarrow \underbrace{-kx_i(t)}_{\text{2nd order ODE}} = m\ddot{x}_i(t)$$

Idea: Simplify One 2nd Order ODE to Two 1st Order ODE



- 2nd order ODE:

$$-kx_i(t) = m\ddot{x}_i(t)$$

- We know that:

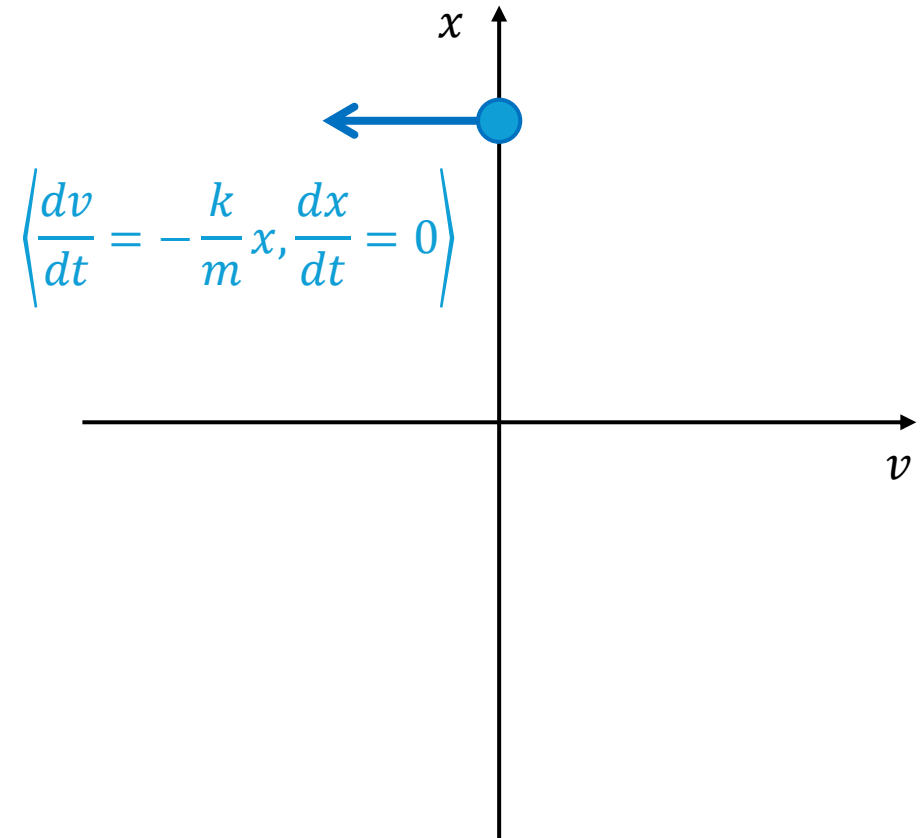
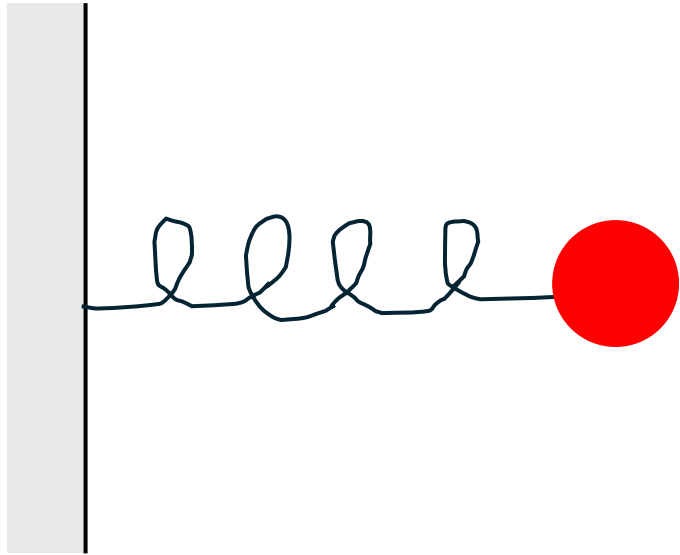
$$\ddot{x} = \dot{v}$$

$$\dot{x} = v$$

- We convert a 2nd order ODE to two 1st order ODEs:

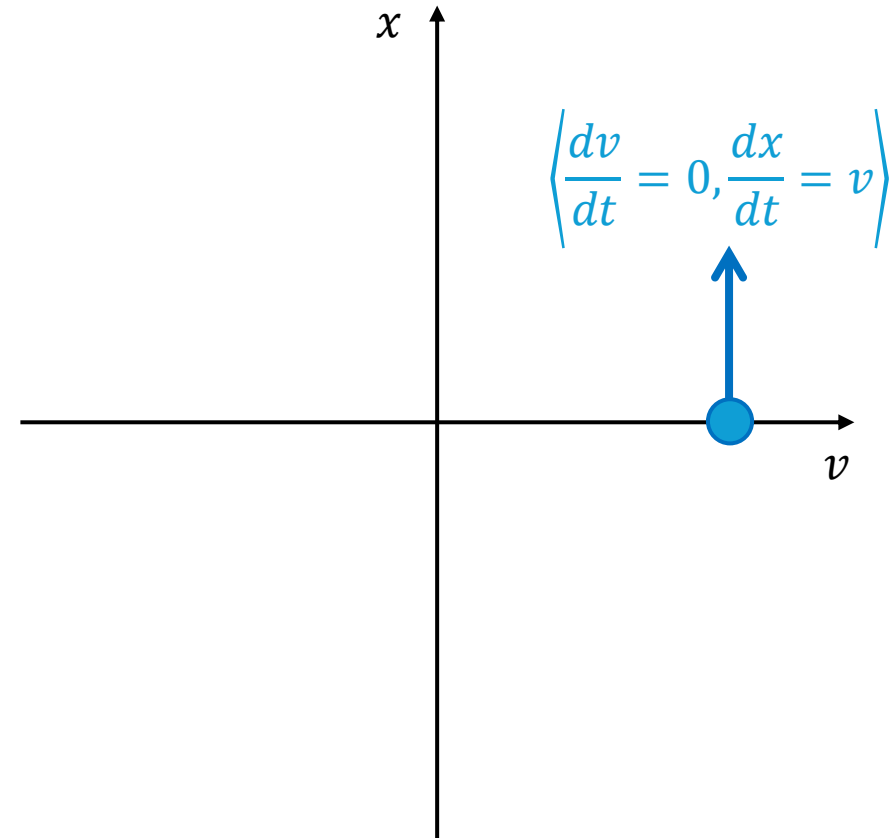
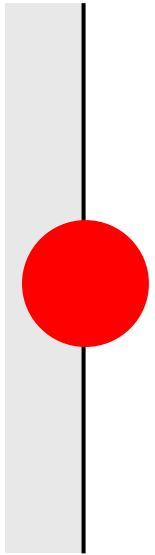
$$\underbrace{\begin{pmatrix} m & 0 \\ 0 & 1 \end{pmatrix}}_A \underbrace{\frac{d}{dt} \begin{pmatrix} v \\ x \end{pmatrix}}_{\dot{y}} = \underbrace{\begin{pmatrix} 0 & -k \\ 1 & 0 \end{pmatrix}}_{f(y)} \underbrace{\begin{pmatrix} v \\ x \end{pmatrix}}_y$$

Idea: Simplify One 2nd Order ODE to Two 1st Order ODE



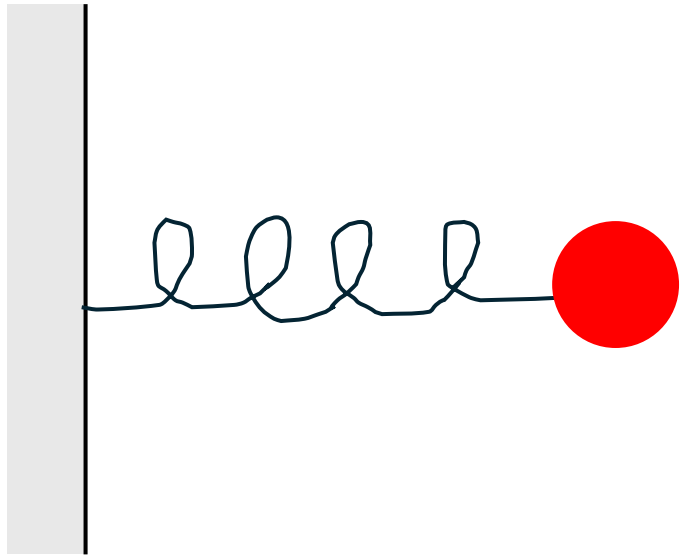
$$\begin{pmatrix} m & 0 \\ 0 & 1 \end{pmatrix} \frac{d}{dt} \begin{pmatrix} v \\ x \end{pmatrix} = \begin{pmatrix} 0 & -k \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 0 \\ x \end{pmatrix}$$

Idea: Simplify One 2nd Order ODE to Two 1st Order ODE



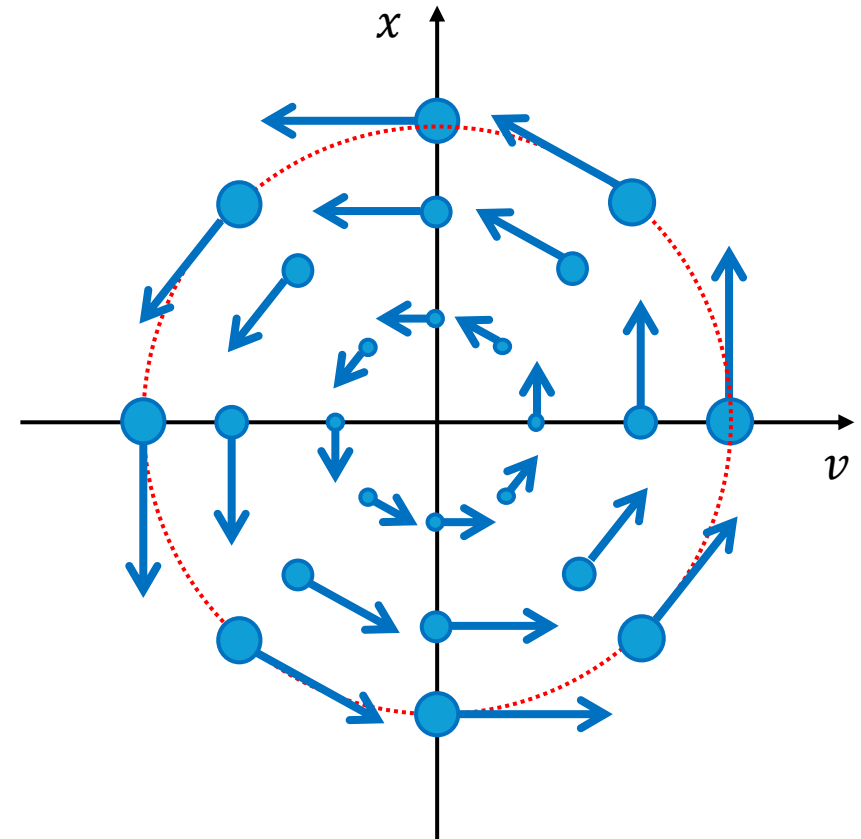
$$\begin{pmatrix} m & 0 \\ 0 & 1 \end{pmatrix} \frac{d}{dt} \begin{pmatrix} v \\ x \end{pmatrix} = \begin{pmatrix} 0 & -k \\ 1 & 0 \end{pmatrix} \begin{pmatrix} v \\ x \end{pmatrix}$$

Idea: Simplify One 2nd Order ODE to Two 1st Order ODE



$$\begin{pmatrix} m & 0 \\ 0 & 1 \end{pmatrix} \frac{d}{dt} \begin{pmatrix} v \\ x \end{pmatrix} = \begin{pmatrix} 0 & -k \\ 1 & 0 \end{pmatrix} \begin{pmatrix} v \\ x \end{pmatrix}$$

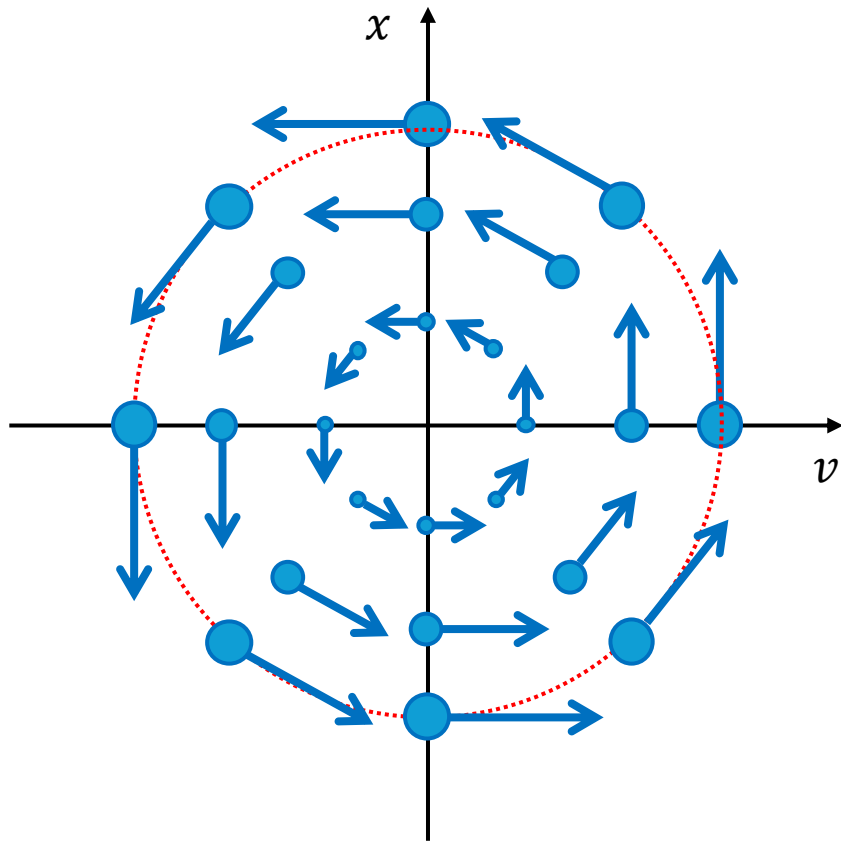
We can't always solve this equation analytically.
What are the numerical approximators?



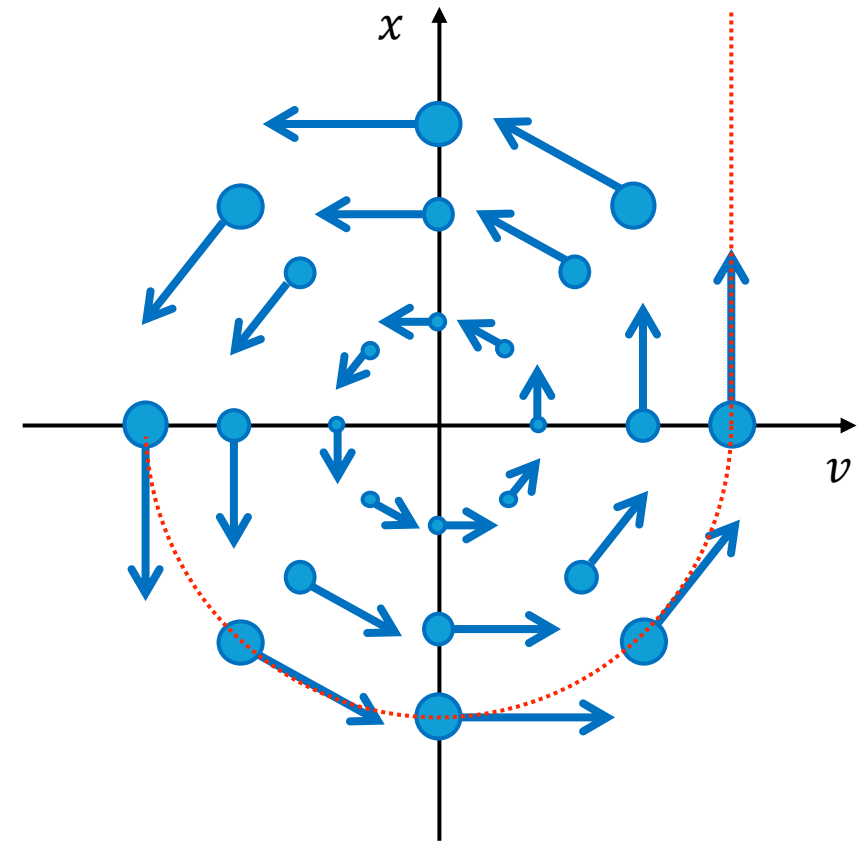
Time Integration Algorithms

- Problem: How to solve the ODE numerically? We can only approximate infinitesimal dt with discrete Δt
- Two types of time integration algorithms:
 1. Explicit time integration methods: Next time step can be computed entirely using values from the current time step or before
 2. Implicit time integration methods: Next time step is computed using values from the future
- How to choose the time integration algorithms?
 - Performance
 - Stability
 - Accuracy

Stability of Time Integration Algorithms

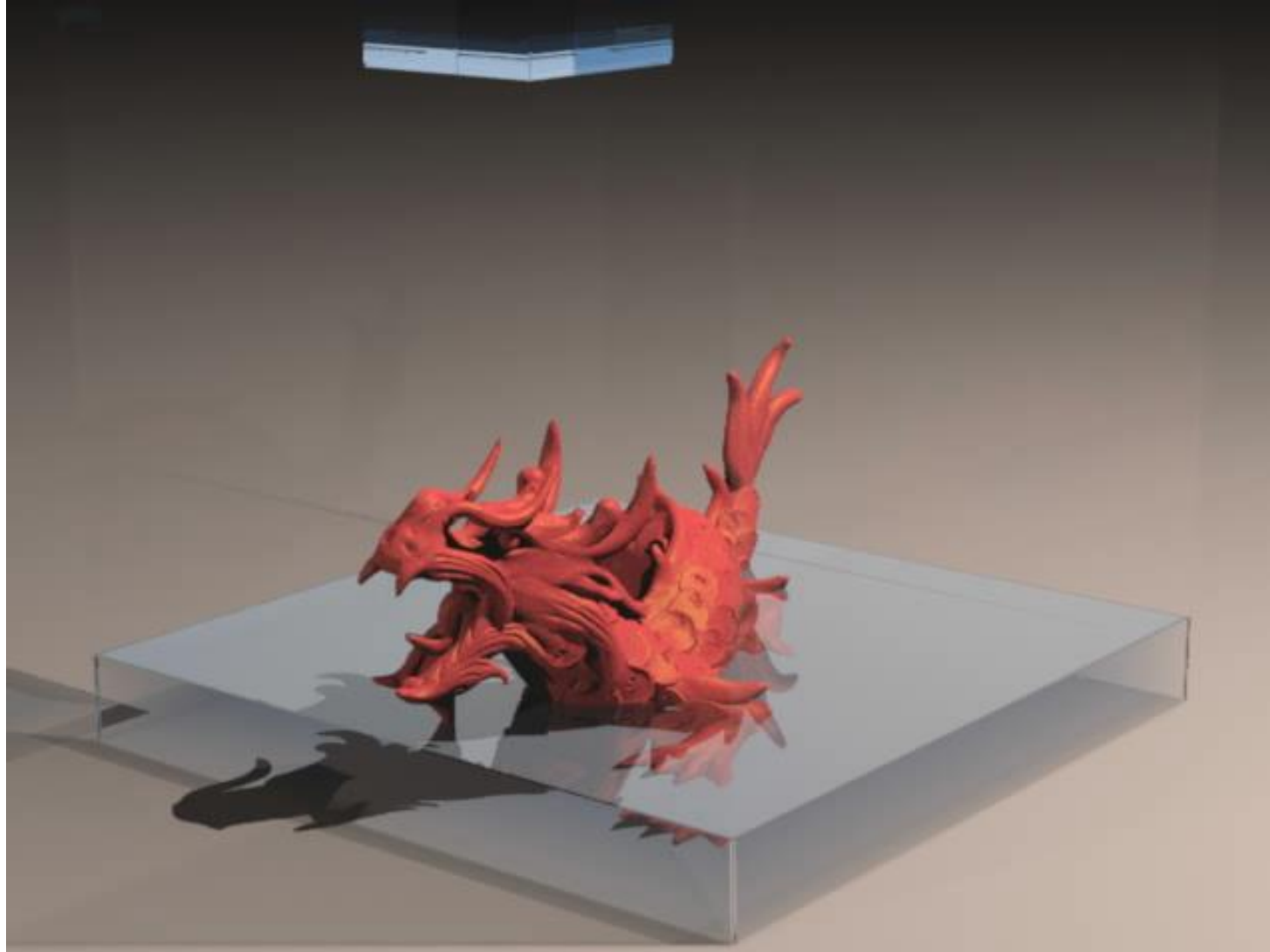


Stable: the path traced out by the numerical approximator stays the same as the analytical solution



Unstable: the path traced out by the numerical approximator diverges from the analytical solution

Accuracy of Time Integration Algorithms



Forward Euler Time Integration

- Our ODE equation:

$$\underbrace{\begin{pmatrix} m & 0 \\ 0 & 1 \end{pmatrix}}_A \underbrace{\frac{d}{dt}}_{\dot{y}} \underbrace{\begin{pmatrix} v \\ x \end{pmatrix}}_{\overset{y}{\boxed{\begin{pmatrix} v \\ x \end{pmatrix}}}} = \underbrace{\begin{pmatrix} 0 & -k \\ 1 & 0 \end{pmatrix}}_{f(y)} \underbrace{\begin{pmatrix} v \\ x \end{pmatrix}}_{\overset{y}{\boxed{\begin{pmatrix} v \\ x \end{pmatrix}}}}$$

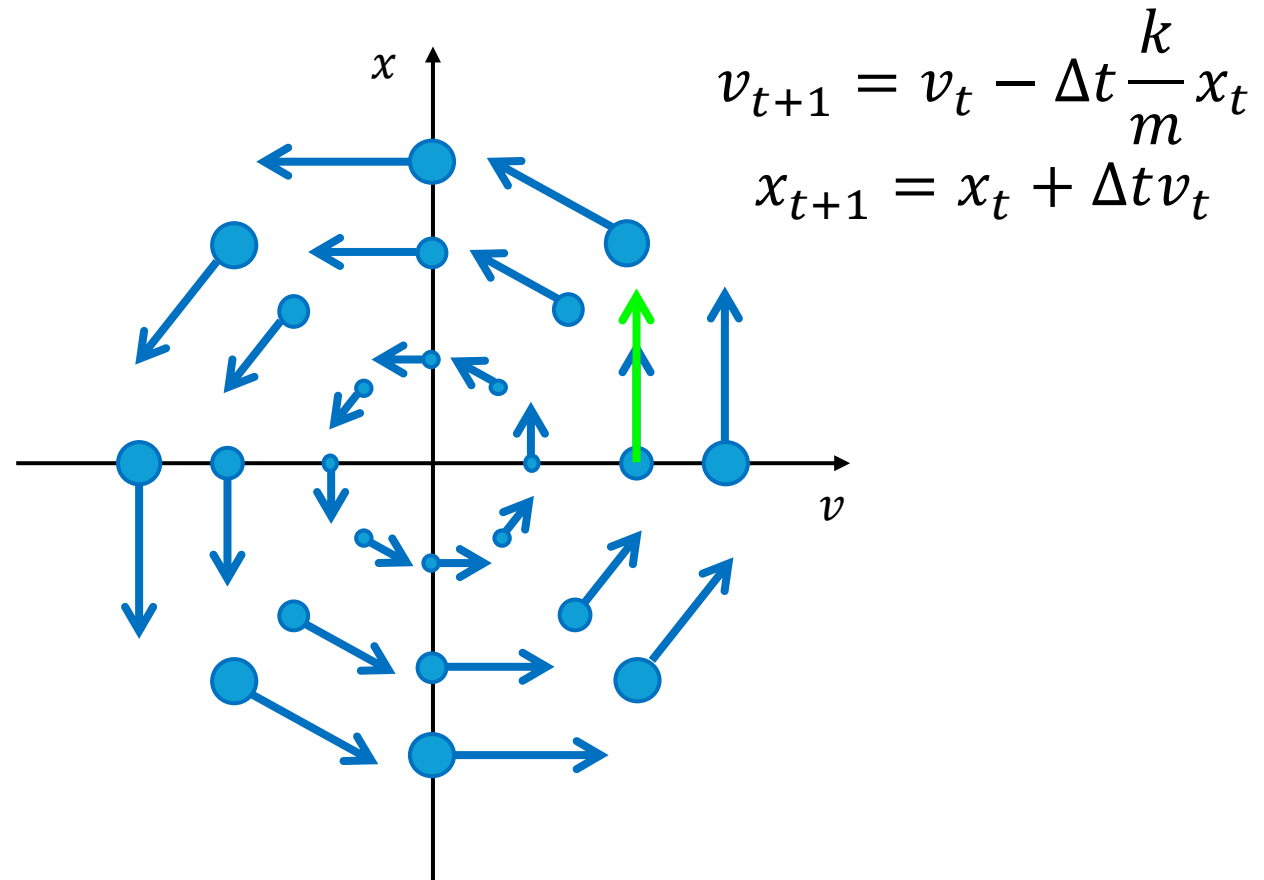
- We have: $\dot{y} \approx \frac{1}{\Delta t} (y_{t+1} - y_t)$
- Substitute $\dot{y}$ in the original ODE:

$$A \frac{1}{\Delta t} (y_{t+1} - y_t) = f(y_t) \Rightarrow y_{t+1} = \underbrace{\frac{\Delta t}{A} f(y_t)}_{\text{blue}} + y_t$$

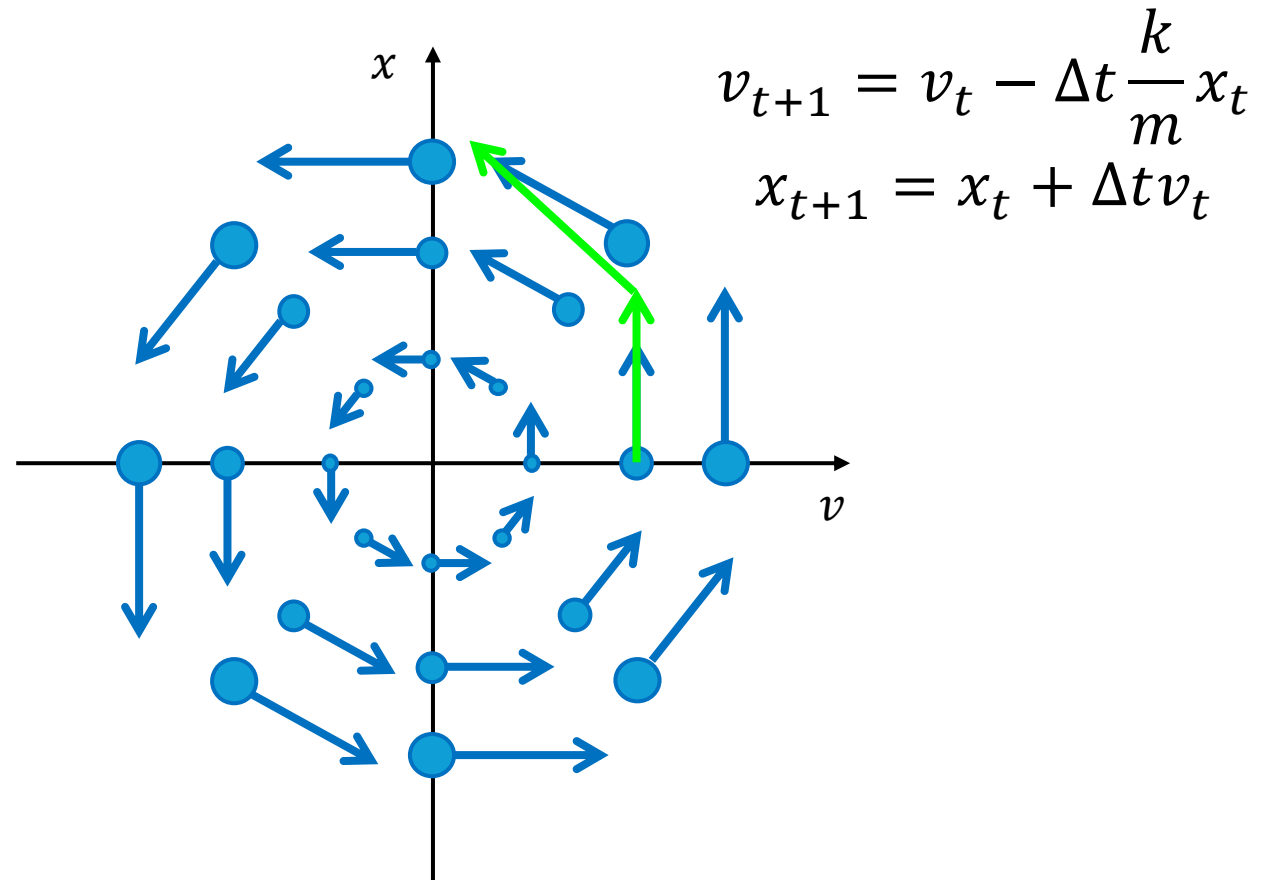
$$v_{t+1} = v_t - \Delta t \frac{k}{m} x_t$$

$$x_{t+1} = x_t + \Delta t v_t$$

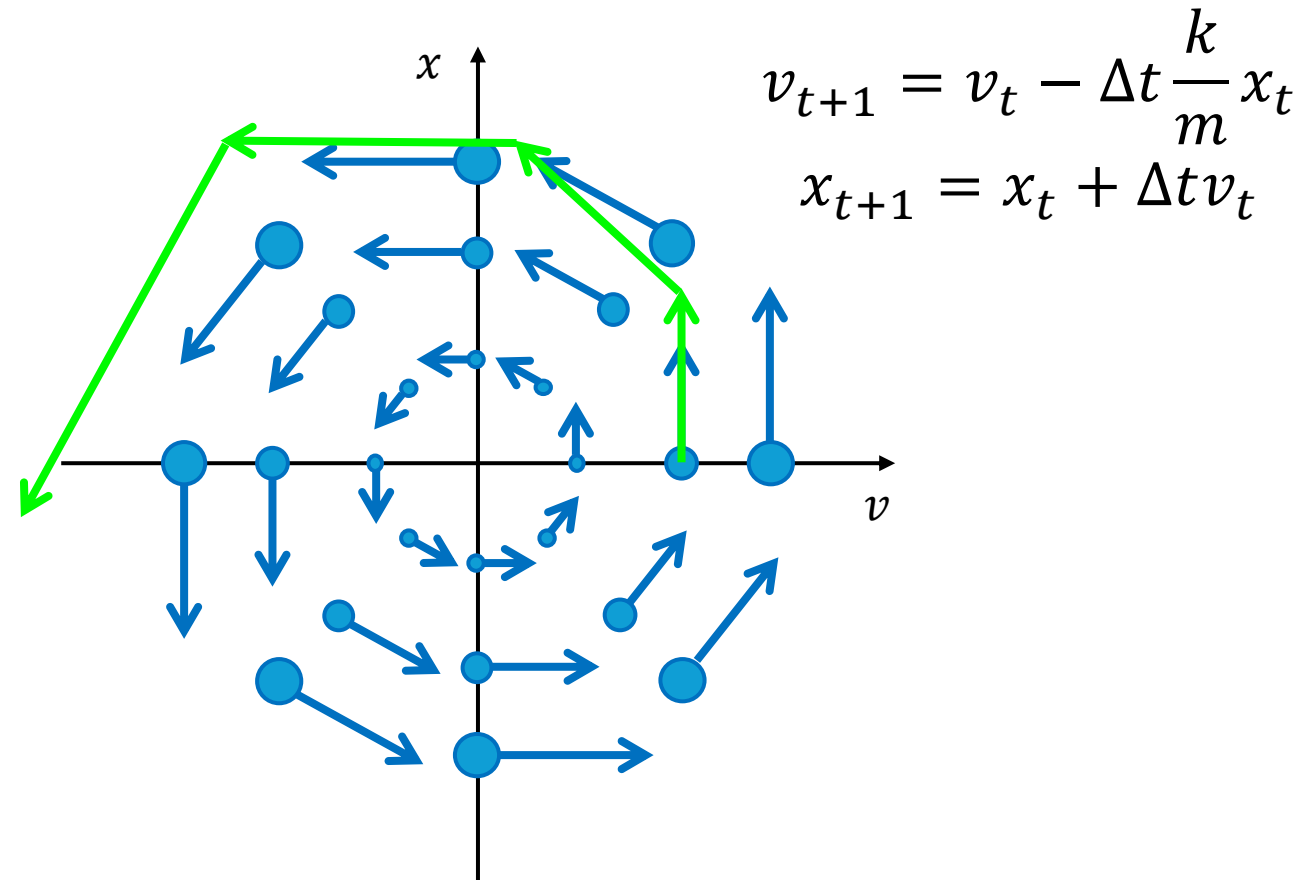
Forward Euler Time Integration



Forward Euler Time Integration



Forward Euler Time Integration

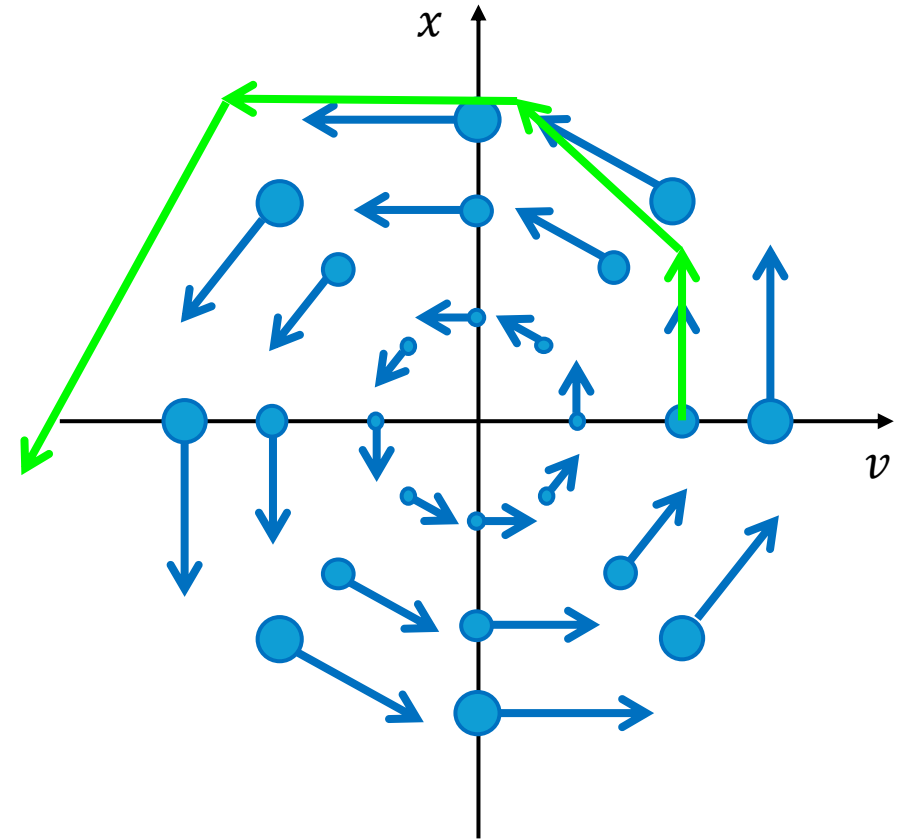


Forward Euler Time Integration

- Forward Euler time integration updates y with “current slope”:

$$y_{t+1} = \frac{\Delta t}{A} f(y_t) + y_t$$

- Problem: the integrated values overshoot as the numerical update lags behind the analytical solution
- Idea:
 - Update y with “average slope” of the current and future slope.



Runge-Kutta Time Integration

- Update $\mathbf{y}$ with “average slope” with real (current/previous) slope $f(\mathbf{y}_{t+a})$ and estimated (future) slope $f(\tilde{\mathbf{y}}_{t+b})$:

averaging coefficients

$$\mathbf{y}_{t+1} = \mathbf{y}_t + \frac{\Delta t}{A} (\alpha f(\mathbf{y}_{t+a}) + \beta f(\tilde{\mathbf{y}}_{t+b}))$$

- Using Forward Euler Time Integration to estimate $\tilde{\mathbf{y}}_{t+b}$:

$$\tilde{\mathbf{y}}_{t+b} = \frac{b\Delta t}{A} f(\mathbf{y}_t) + \mathbf{y}_t$$

Heun's Method: A Special Case of Runge-Kutta Time Integration

- Update $\mathbf{y}$ with “average slope” with the current slope $f(\mathbf{y}_t)$ and estimated future slope $f(\tilde{\mathbf{y}}_{t+1})$:

$$\mathbf{y}_{t+1} = \mathbf{y}_t + \frac{\Delta t}{A} \left(\frac{1}{2} f(\mathbf{y}_t) + \frac{1}{2} f(\tilde{\mathbf{y}}_{t+1}) \right)$$

- Using Forward Euler Time Integration to estimate $\tilde{\mathbf{y}}_{t+1}$:

$$\tilde{\mathbf{y}}_{t+1} = \frac{\Delta t}{A} f(\mathbf{y}_t) + \mathbf{y}_t$$

Generalization of Runge-Kutta Time Integration

$$\kappa_1 = \mathbf{A}^{-1} \mathbf{f}(\mathbf{y}^t)$$

$$\kappa_2 = \mathbf{A}^{-1} \mathbf{f}(\mathbf{y}^t + \Delta t \cdot \kappa_1)$$

$$\mathbf{y}^{t+1} = \mathbf{y}^t + \frac{\Delta t}{2} (\kappa_1 + \kappa_2)$$

Generalization of Runge-Kutta Time Integration

Fourth-Order Runge Kutta

$$\kappa_1 = A^{-1} \mathbf{f}(\mathbf{y}^t)$$

$$\kappa_2 = A^{-1} \mathbf{f} \left(\mathbf{y}^t + \frac{\Delta t}{2} \cdot \kappa_1 \right)$$

$$\kappa_3 = A^{-1} \mathbf{f} \left(\mathbf{y}^t + \frac{\Delta t}{2} \cdot \kappa_2 \right)$$

$$\kappa_4 = A^{-1} \mathbf{f}(\mathbf{y}^t + \Delta t \cdot \kappa_3)$$

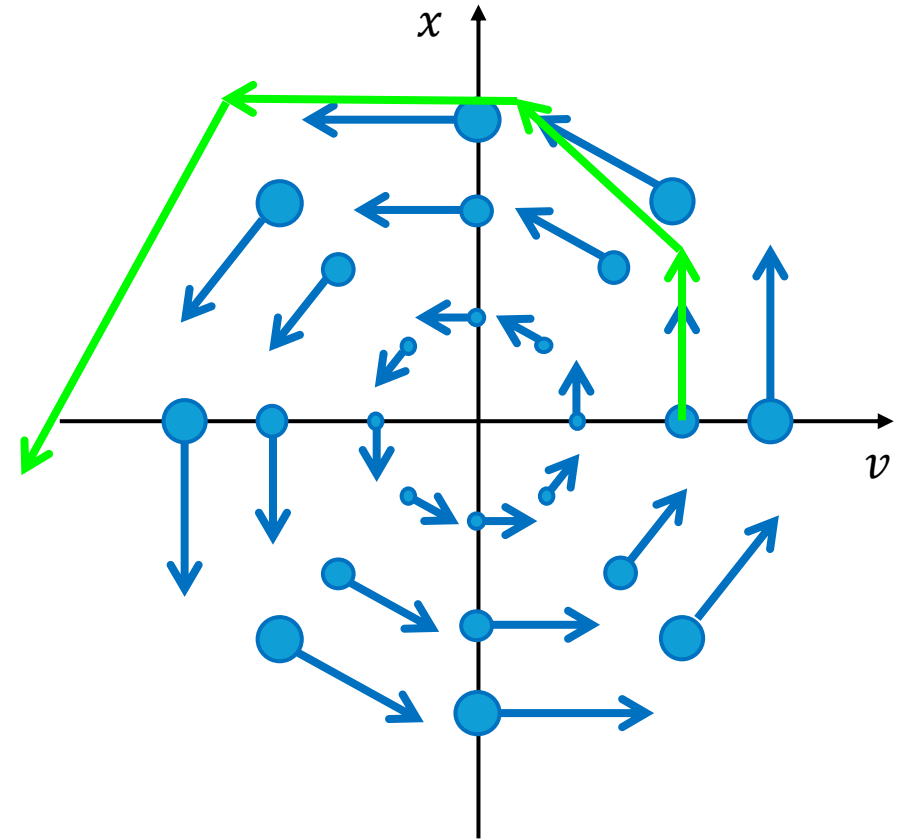
$$\mathbf{y}^{t+1} = \mathbf{y}^t + \frac{\Delta t}{6} (\kappa_1 + 2 \cdot \kappa_2 + 2 \cdot \kappa_3 + \kappa_4)$$

Forward Euler Time Integration

- Forward Euler time integration updates y with “current slope”:

$$y_{t+1} = \frac{\Delta t}{A} f(y_t) + y_t$$

- Problem: the integrated values overshoot as the numerical update lags behind the analytical solution
- Idea:
 - Update y with “average slope” of the current and future slope.
 - Update y with “future slope”



Backward Euler Time Integration

- Our ODE equation:

$$\underbrace{\begin{pmatrix} m & 0 \\ 0 & 1 \end{pmatrix}}_A \underbrace{\frac{d}{dt} \begin{pmatrix} v \\ x \end{pmatrix}}_{\dot{y}} = \underbrace{\begin{pmatrix} 0 & -k \\ 1 & 0 \end{pmatrix}}_{f(y)} \underbrace{\begin{pmatrix} v \\ x \end{pmatrix}}_y$$

- We have: $\dot{y} \approx \frac{1}{\Delta t} (y_{t+1} - y_t)$
- The original ODE can be re-written as:

$$A \frac{1}{\Delta t} (y_{t+1} - y_t) = f(y_{t+1})$$

Update y with future slope

Backward Euler Time Integration

- Our ODE equation:

$$\underbrace{\begin{pmatrix} m & 0 \\ 0 & 1 \end{pmatrix}}_A \underbrace{\frac{d}{dt} \begin{pmatrix} v \\ x \end{pmatrix}}_{\dot{y}} = \underbrace{\begin{pmatrix} 0 & -k \\ 1 & 0 \end{pmatrix}}_{f(y)} \underbrace{\begin{pmatrix} v \\ x \end{pmatrix}}_y$$

- We have: $\dot{y} \approx \frac{1}{\Delta t} (y_{t+1} - y_t)$
- The original ODE can be re-written as:

$$\begin{aligned} A \frac{1}{\Delta t} (y_{t+1} - y_t) &= f(y_{t+1}) \Rightarrow y_{t+1} = \frac{\Delta t}{A} B y_{t+1} + y_t \\ &\Rightarrow \left(I - \frac{\Delta t}{A} B \right) y_{t+1} = y_t \end{aligned}$$

Backward Euler Time Integration

- How to solve $\left(I - \frac{\Delta t}{A} B\right) y_{t+1} = y_t$?
- We have:

$$v_{t+1} + \Delta t \frac{k}{m} x_{t+1} = v_t$$
$$x_{t+1} - \Delta t v_{t+1} = x_t$$

- Substitute x_{t+1} into the first equation, we have:

$$v_{t+1} + \Delta t \frac{k}{m} (x_t + \Delta t v_{t+1}) = v_t$$
$$\Rightarrow \left(1 + \Delta t^2 \frac{k}{m}\right) v_{t+1} = v_t - \Delta t \frac{k}{m} x_t$$

Backward Euler Time Integration

- How to solve $\left(I - \frac{\Delta t}{A} B\right) y_{t+1} = y_t$?
- We have:

$$v_{t+1} + \Delta t \frac{k}{m} x_{t+1} = v_t$$

$$x_{t+1} - \Delta t v_{t+1} = x_t$$

- Substitute x_{t+1} into the first equation, we have:

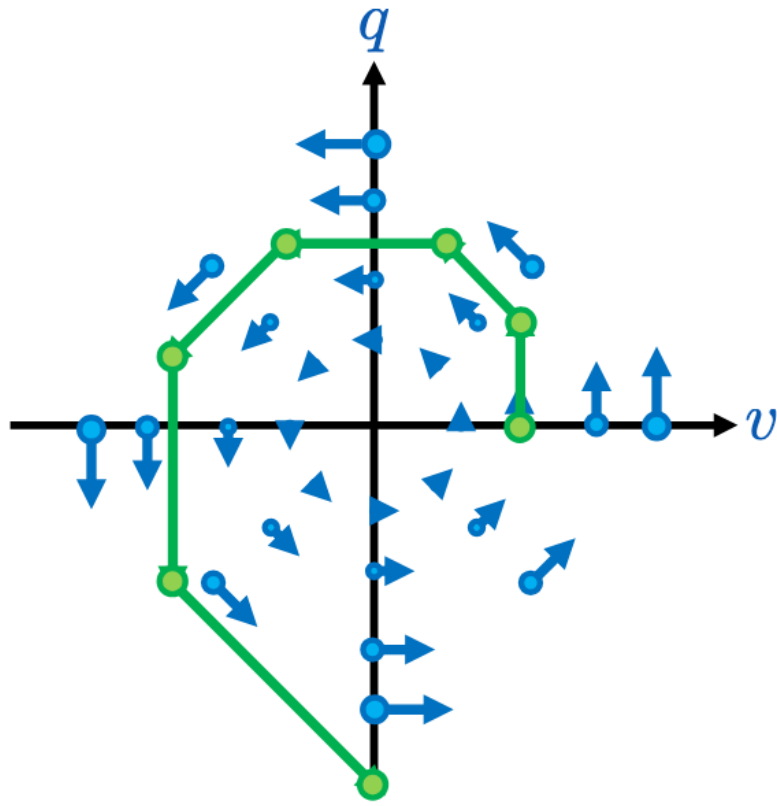
$$v_{t+1} + \Delta t \frac{k}{m} (x_t + \Delta t v_{t+1}) = v_t$$

$$\Rightarrow \left(1 + \Delta t^2 \frac{k}{m}\right) v_{t+1} = v_t - \Delta t \frac{k}{m} x_t$$

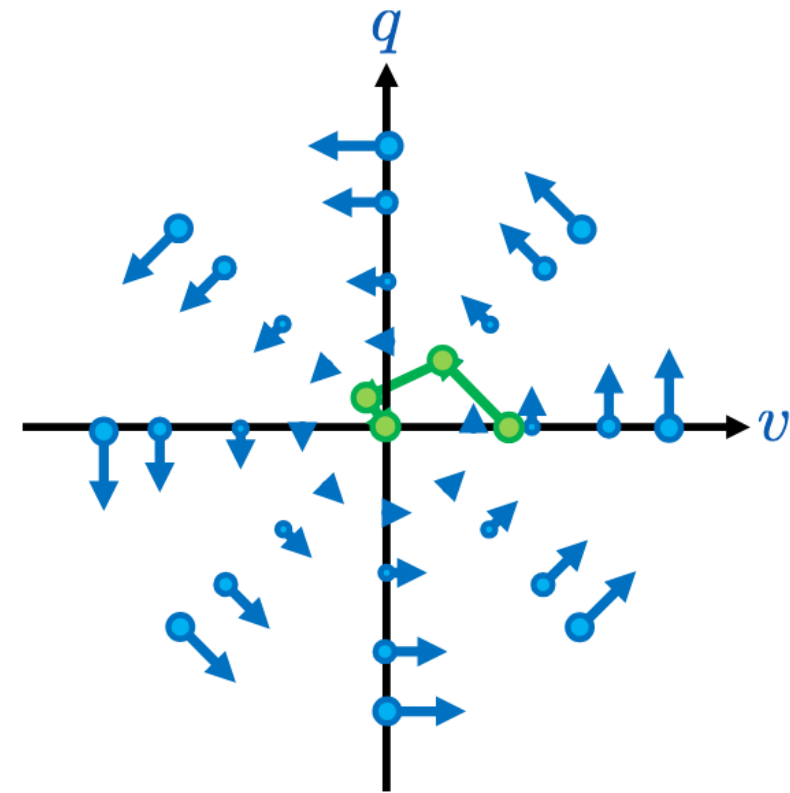
We solve the backward integration implicitly!

Forward vs. Backward Euler Time Integration

- Can we cancel out the exploding and damping effect? Yes! Let's try to mix these two integration methods.



Explicit (exploding!!!)



Implicit (damping)

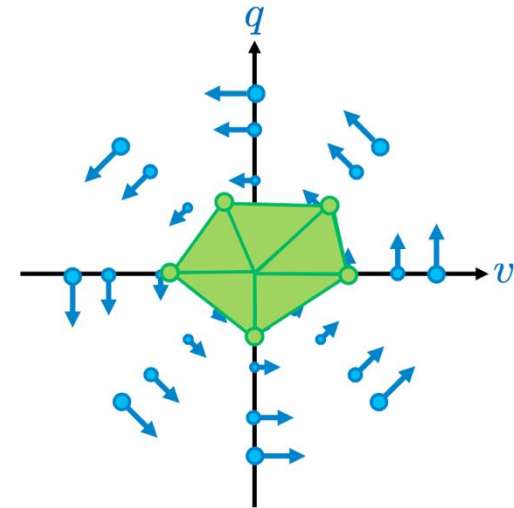
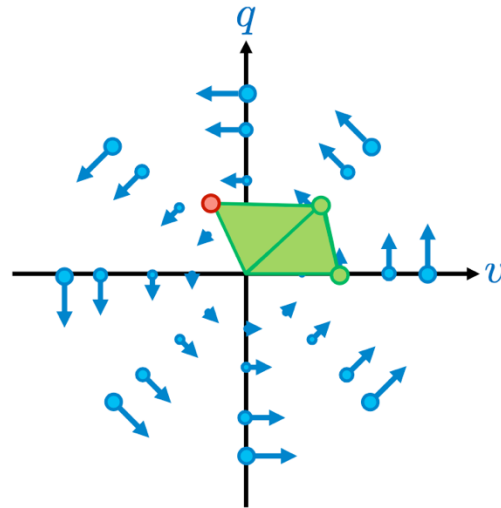
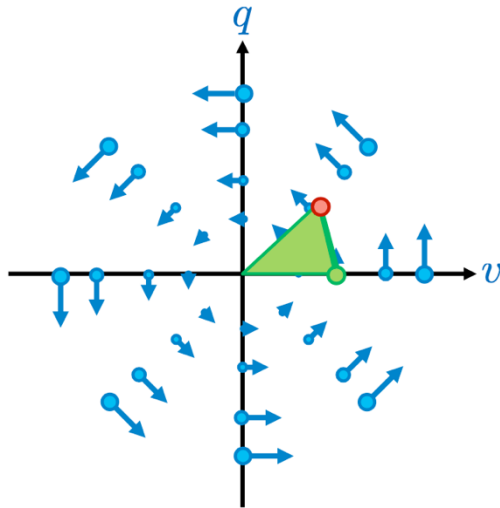
Symplectic Euler Time Integration

- First, take an explicit step

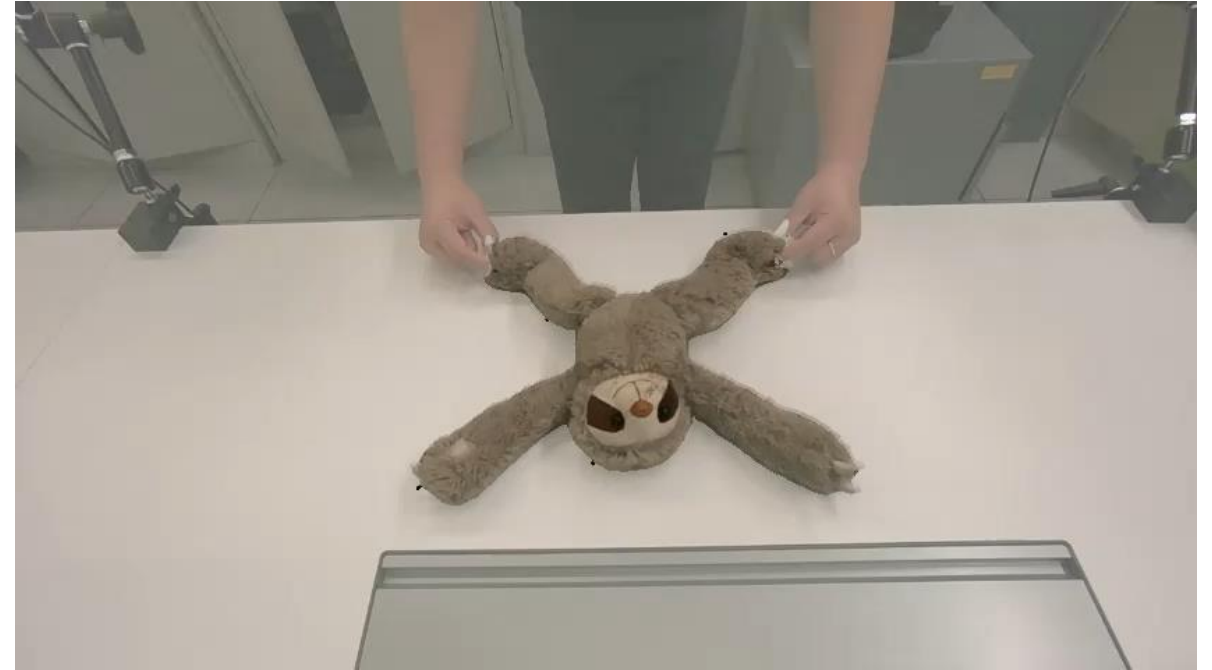
$$v_{t+1} = v_t - \Delta t \frac{k}{m} x_t$$

- Next, take an implicit step

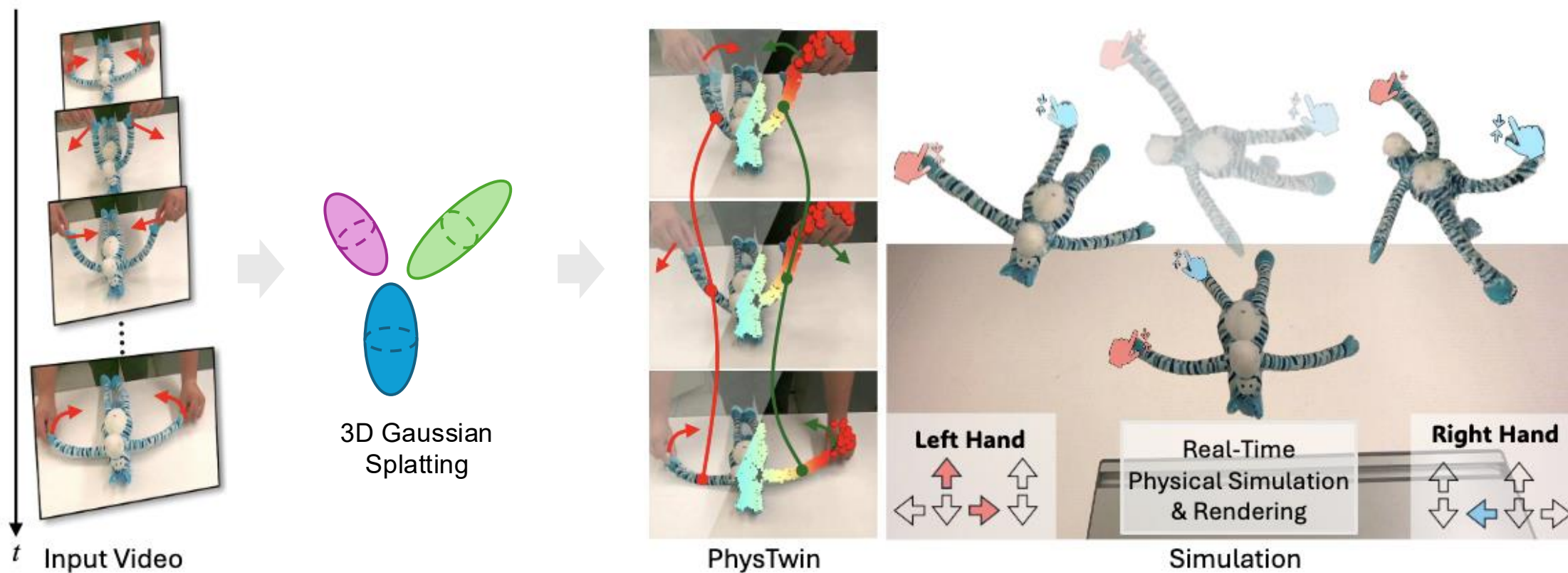
$$x_{t+1} = x_t - \Delta t v_{t+1}$$



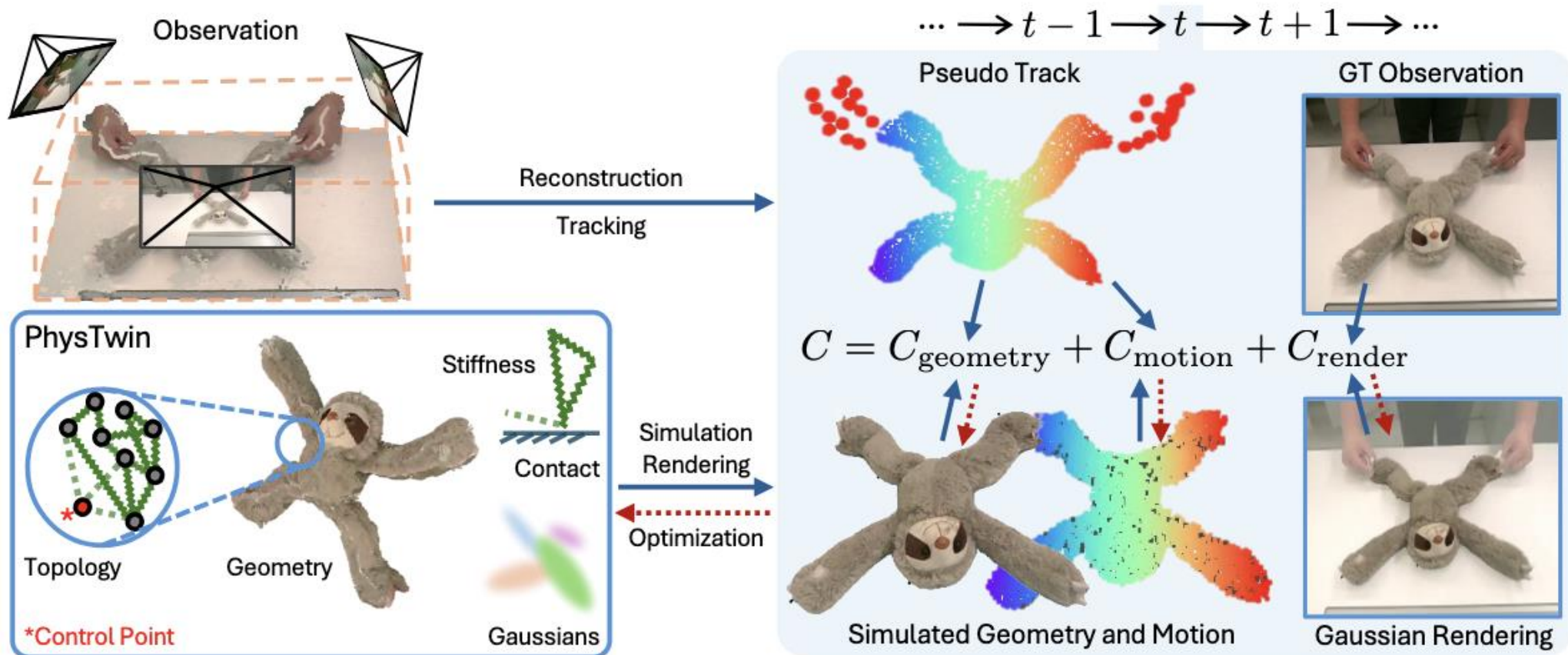
Scene Reconstruction and Simulation with Mass-Spring System



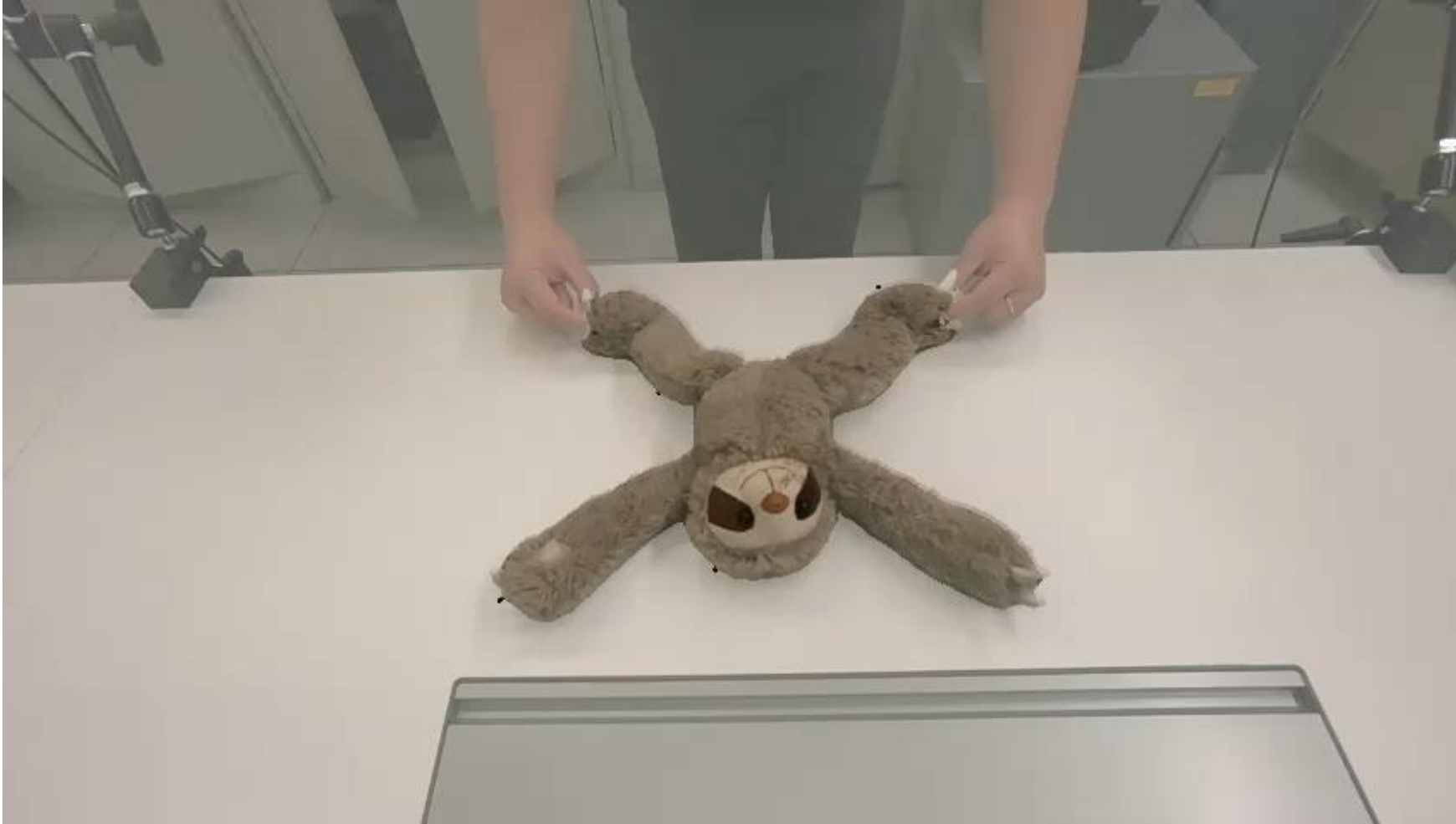
Idea: Integrate 3D Gaussian Splatting with Mass-Spring Modeling



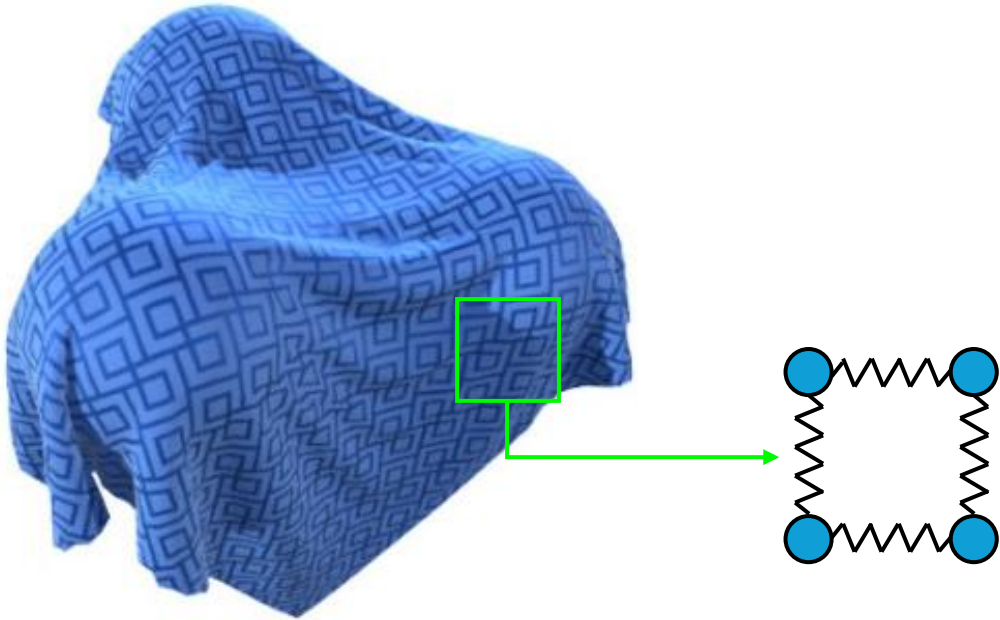
PhysTwin: Physics-Informed Reconstruction and Simulation of Deformable Objects from Videos



Results: Real-to-Sim Demo



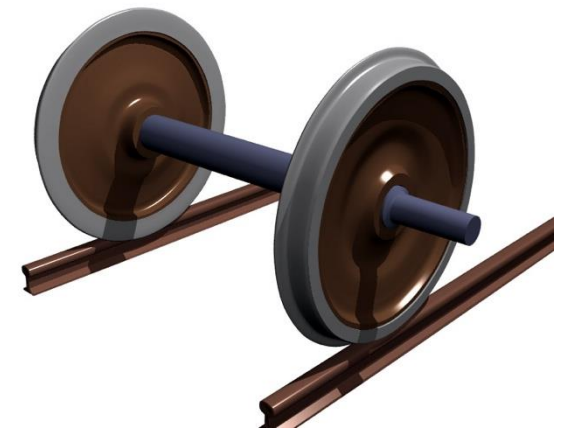
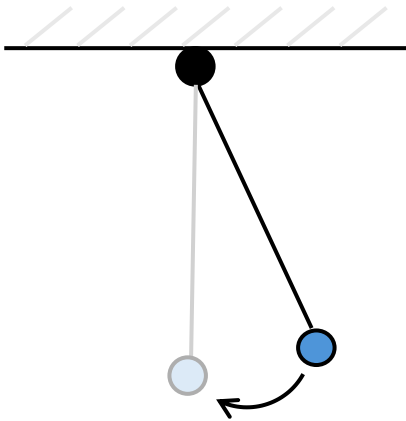
Mass-Spring System Has Problems



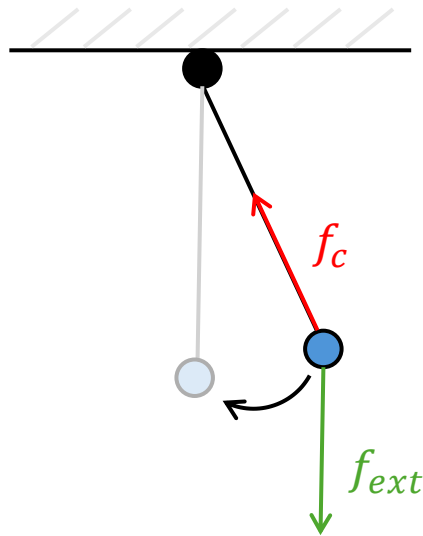
- How to describe objects of different kinetic constraints, e.g. bending, impenetrability etc?
- How to model continuum material? The empty space between spring-connected nodes has no mass and energy.

A Dynamic System May Have Multiple Constraints

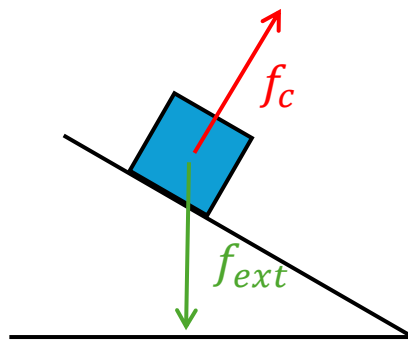
- Constraints limit the motion of objects by restricting them to follow a specific trajectory or path. For example:
 - The circular motion of a pendulum
 - The distance constraints between two objects
 - The resistance when bending an iron rod



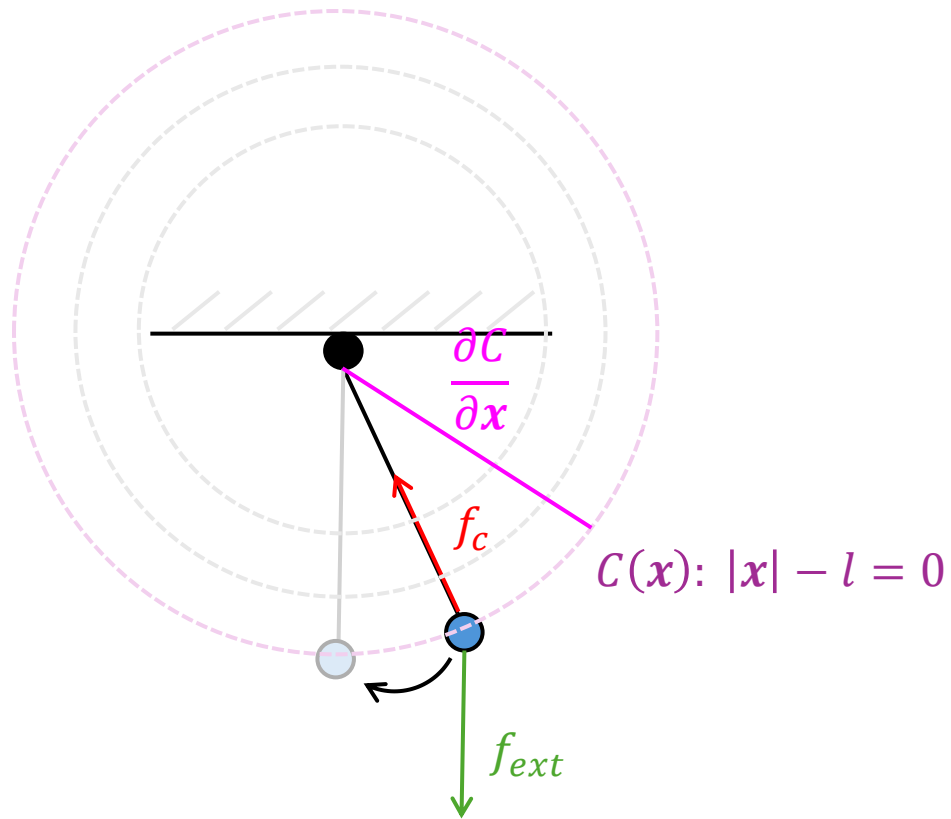
How to Solve a Dynamic System with Constraints?



- Idea 1: Directly calculate the forces required to maintain the constraints
- The job of these constraint forces is to cancel just those parts of the applied forces that act against the constraints



How to Solve a Dynamic System with Constraints?



- Idea 1: Directly calculate the forces required to maintain the constraints
- The job of these constraint forces is to cancel just those parts of the applied forces that act against the constraints
- For a constraint C , the corresponding constraint force is $f_c = \lambda \frac{\partial C}{\partial x}$ where x denotes the spatial coordinates of the dynamic system

Check “Physically Based Modeling: Principles and Practice Constrained Dynamics by Andrew Witkin” for more details

How to Solve a Dynamic System with Constraints?

- Idea 2: Calculate the kinetic update with Newton's law and refine the kinetic update to follow the constraints
- Problem formulation: Given a set of positions $\mathbf{x} = [x_1, \dots, x_N]$ and constraints $C_1(\mathbf{x}), \dots, C_M(\mathbf{x})$, the goal is to find new positions $\mathbf{p} = [p_1, \dots, p_N]$ such that $C_i(\mathbf{p}) \geq 0 \forall i$

A Transition Slide to PBD

- TODO

Position Based Dynamics

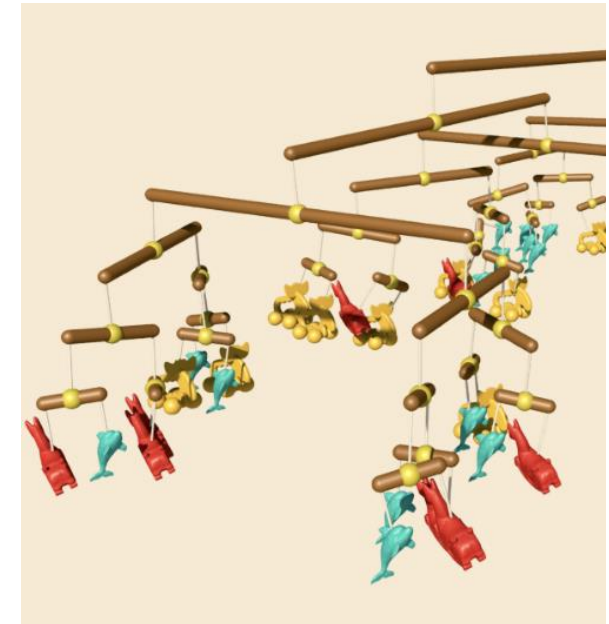
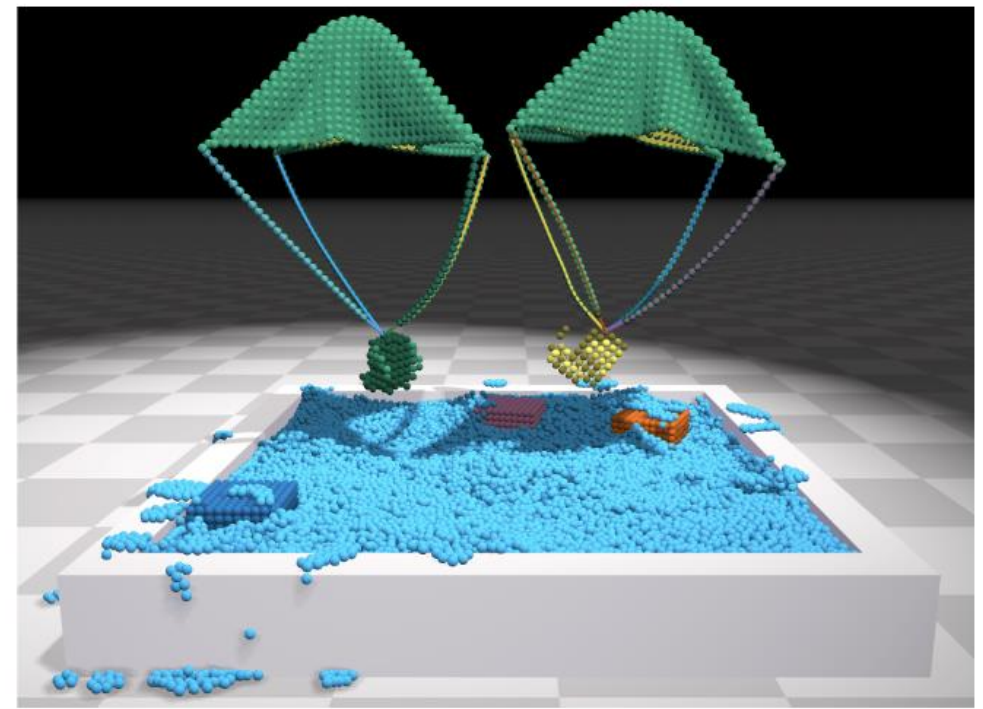
Algorithm 1 Position-based dynamics

```
1: for all vertices  $i$  do
2:   initialize  $\mathbf{x}_i = \mathbf{x}_i^0$ ,  $\mathbf{v}_i = \mathbf{v}_i^0$ ,  $w_i = 1/m_i$ 
3: end for
4: loop           Derive the initial kinetic update with Newton's law
5:   for all vertices  $i$  do  $\mathbf{v}_i \leftarrow \mathbf{v}_i + \Delta t w_i \mathbf{f}_{\text{ext}}(\mathbf{x}_i)$ 
6:   for all vertices  $i$  do  $\mathbf{p}_i \leftarrow \mathbf{x}_i + \Delta t \mathbf{v}_i$ 
7:   for all vertices  $i$  do genCollConstraints( $\mathbf{x}_i \rightarrow \mathbf{p}_i$ )
8:   loop solverIteration times
9:     projectConstraints( $C_1, \dots, C_{M+M_{\text{Coll}}}, \mathbf{p}_1, \dots, \mathbf{p}_N$ )
10:  end loop       Modify the kinetic update based on constraints
11:  for all vertices  $i$  do
12:     $\mathbf{v}_i \leftarrow (\mathbf{p}_i - \mathbf{x}_i) / \Delta t$ 
13:     $\mathbf{x}_i \leftarrow \mathbf{p}_i$ 
14:  end for
15:  velocityUpdate( $\mathbf{v}_1, \dots, \mathbf{v}_N$ )
16: end loop
```

Position Based Dynamics

Algorithm 1 Position-based dynamics

```
1: for all vertices  $i$  do
2:   initialize  $\mathbf{x}_i = \mathbf{x}_i^0$ ,  $\mathbf{v}_i = \mathbf{v}_i^0$ ,  $w_i = 1/m_i$ 
3: end for
4: loop           Derive the initial kinetic update with Newton's law
5:   for all vertices  $i$  do  $\mathbf{v}_i \leftarrow \mathbf{v}_i + \Delta t w_i \mathbf{f}_{\text{ext}}(\mathbf{x}_i)$ 
6:   for all vertices  $i$  do  $\mathbf{p}_i \leftarrow \mathbf{x}_i + \Delta t \mathbf{v}_i$ 
7:   for all vertices  $i$  do genCollConstraints( $\mathbf{x}_i \rightarrow \mathbf{p}_i$ )
8:   loop solverIteration times
9:     projectConstraints( $C_1, \dots, C_{M+M_{\text{Coll}}}, \mathbf{p}_1, \dots, \mathbf{p}_N$ )
10:  end loop       Modify the kinetic update based on constraints
11:  for all vertices  $i$  do
12:     $\mathbf{v}_i \leftarrow (\mathbf{p}_i - \mathbf{x}_i) / \Delta t$ 
13:     $\mathbf{x}_i \leftarrow \mathbf{p}_i$ 
14:  end for
15:  velocityUpdate( $\mathbf{v}_1, \dots, \mathbf{v}_N$ )
16: end loop
```



The idea is applicable to both particles and rigid bodies

Constraint Projection

- Let $\mathbf{p} = \mathbf{x} + \Delta\mathbf{x}$. The problem becomes to find the correction $\Delta\mathbf{x}$ such that $C(\mathbf{x} + \Delta\mathbf{x}) > 0$.
- We can first approximate the constraint equation with Taylor expansion:

$$C(\mathbf{x} + \Delta\mathbf{x}) \approx C(\mathbf{x}) + \nabla C(\mathbf{x}) \cdot \Delta\mathbf{x} > 0 \quad \nabla = \left(\frac{\partial}{\partial x_1}, \dots, \frac{\partial}{\partial x_N} \right)$$

- A possibly good solution is to restrict $\Delta\mathbf{x}$ to be in the direction of $\nabla C(\mathbf{x})$.
Let $\mathbf{M} = \text{diag}(m_1, \dots, m_N)$ be the diagonal matrix of point masses

$$\Delta\mathbf{x} = \lambda \mathbf{M}^{-1} \nabla C(\mathbf{x})^\top$$

Why this is a possibly good solution?

Constraint Projection

- We can first approximate the constraint equation with Taylor expansion:

$$C(\mathbf{x} + \Delta\mathbf{x}) \approx C(\mathbf{x}) + \nabla C(\mathbf{x}) \cdot \Delta\mathbf{x} > 0$$

- A possibly good solution is to restrict $\Delta\mathbf{x}$ to be in the direction of $\nabla C(\mathbf{x})$.
Let $\mathbf{M} = \text{diag}(m_1, \dots, m_N)$ be the diagonal matrix of point masses

$$\Delta\mathbf{x} = \lambda \mathbf{M}^{-1} \nabla C(\mathbf{x})^\top$$

1. Linear / angular momentum is conserved: No position / velocity change along the constrained motion direction
2. The value of $C(\mathbf{x} + \Delta\mathbf{x})$ will increase

Constraint Projection

- A possibly good solution is to restrict $\Delta \mathbf{x}$ to be in the direction of $\nabla C(\mathbf{x})$. Let $M = \text{diag}(m_1, \dots, m_N)$ be the diagonal matrix of point masses

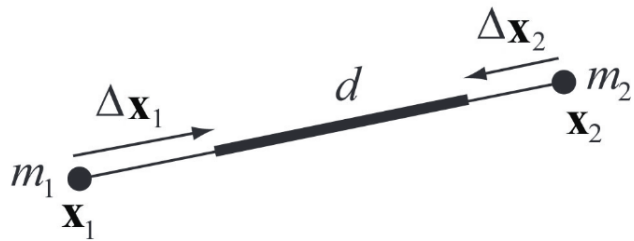
$$\Delta \mathbf{x} = \lambda M^{-1} \nabla C(\mathbf{x})^\top$$

- Substitute the solution back to $C(\mathbf{x} + \Delta \mathbf{x}) \approx C(\mathbf{x}) + \nabla C(\mathbf{x}) \cdot \Delta \mathbf{x} \succ 0$, we obtain the solution of λ :

$$\lambda = \frac{C(\mathbf{x})}{\nabla C(\mathbf{x}) M^{-1} \nabla C(\mathbf{x})^\top}$$

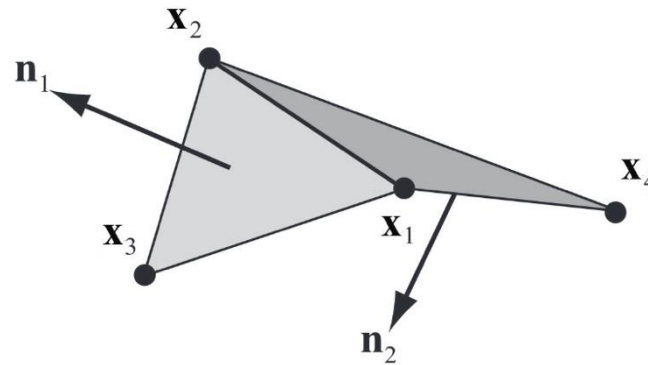
Different Types of Constraints

Stretching



$$C(x_1, x_2) = |x_1 - x_2| - d$$

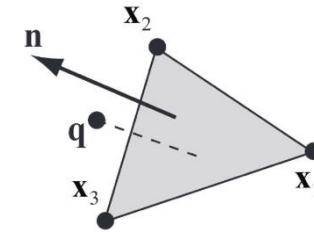
Bending



$$C_{bend}(\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3, \mathbf{x}_4) = \arccos \left(\frac{\mathbf{x}_{2,1} \times \mathbf{x}_{3,1}}{|\mathbf{x}_{2,1} \times \mathbf{x}_{3,1}|} \cdot \frac{\mathbf{x}_{2,1} \times \mathbf{x}_{4,1}}{|\mathbf{x}_{2,1} \times \mathbf{x}_{4,1}|} \right) - \varphi_0$$

where $x_{i,j} = x_i - x_j$ and φ_0 is the initial dihedral angle between the two triangles

Collision



$$C(\mathbf{q}, \mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3) = (\mathbf{q} - \mathbf{x}_1) \cdot \frac{\mathbf{x}_{2,1} \times \mathbf{x}_{3,1}}{|\mathbf{x}_{2,1} \times \mathbf{x}_{3,1}|} - h,$$

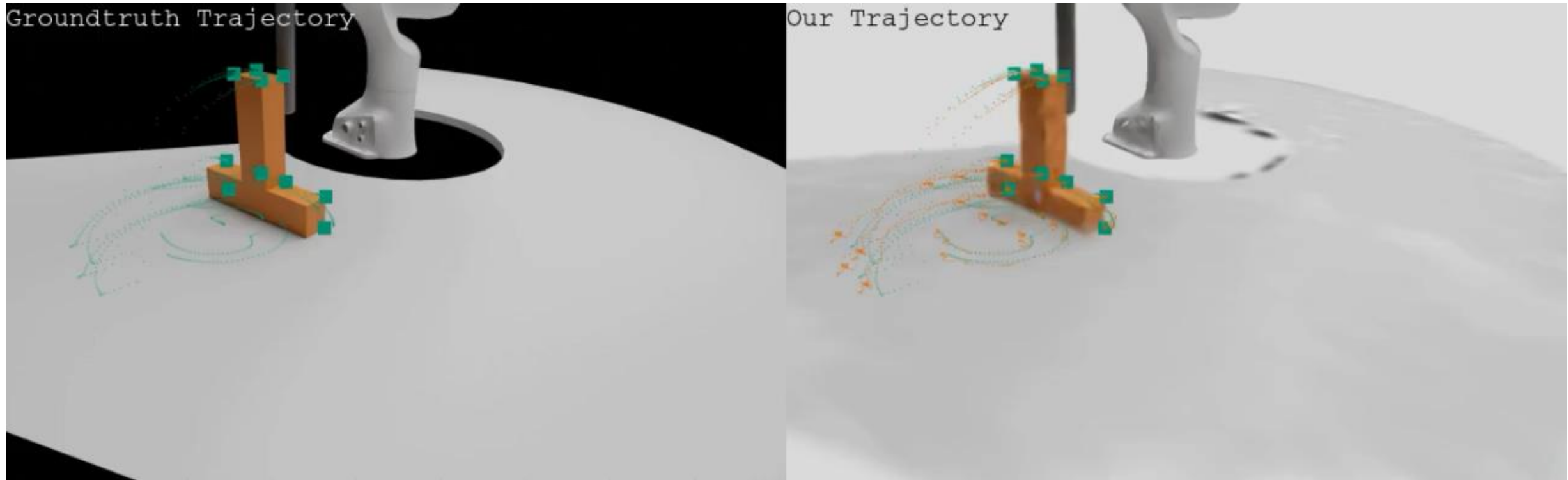
or

$$C(\mathbf{q}, \mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3) = (\mathbf{q} - \mathbf{x}_1) \cdot \frac{\mathbf{x}_{3,1} \times \mathbf{x}_{2,1}}{|\mathbf{x}_{3,1} \times \mathbf{x}_{2,1}|} - h.$$

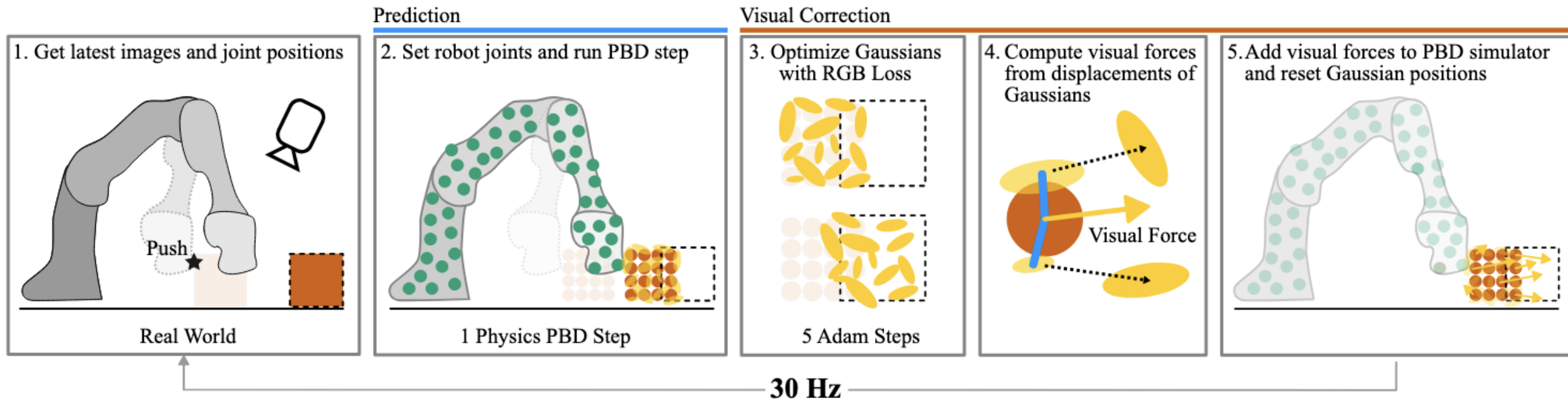
Physical Modeling with Position Based Dynamics



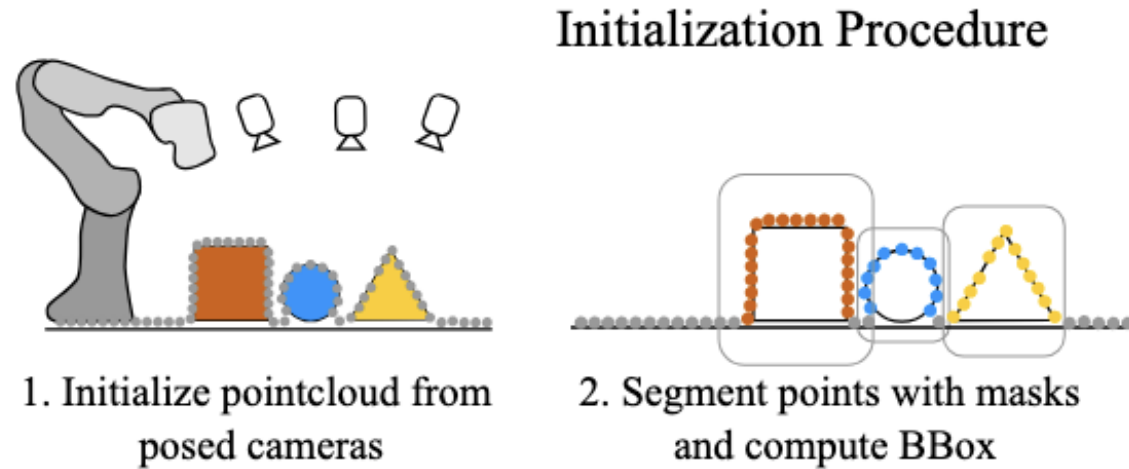
Scene Reconstruction and Simulation with Position-Based Dynamics



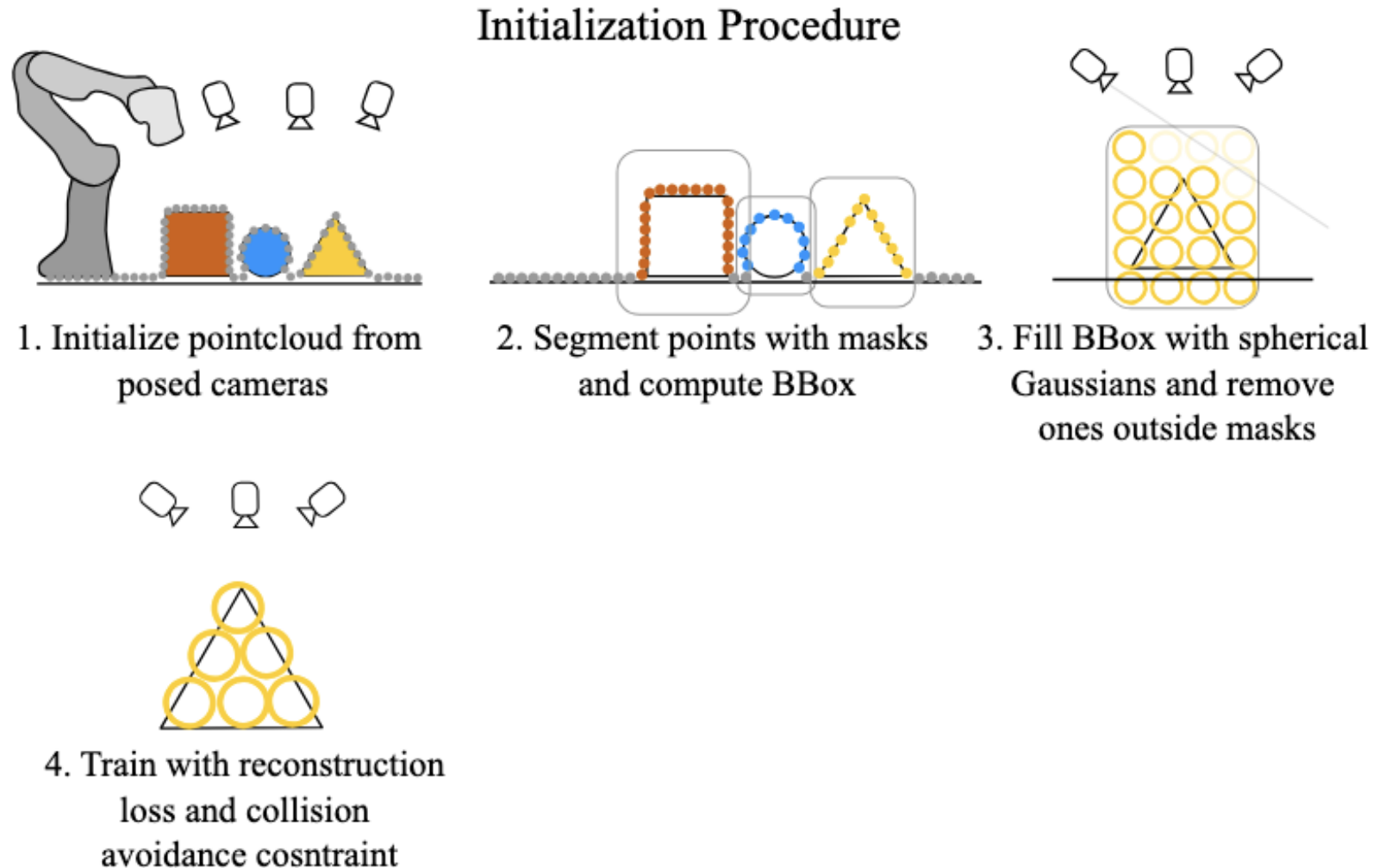
Idea: Integrate 3D Gaussian Splatting with PBD System and Correct Simulation with Visual Input



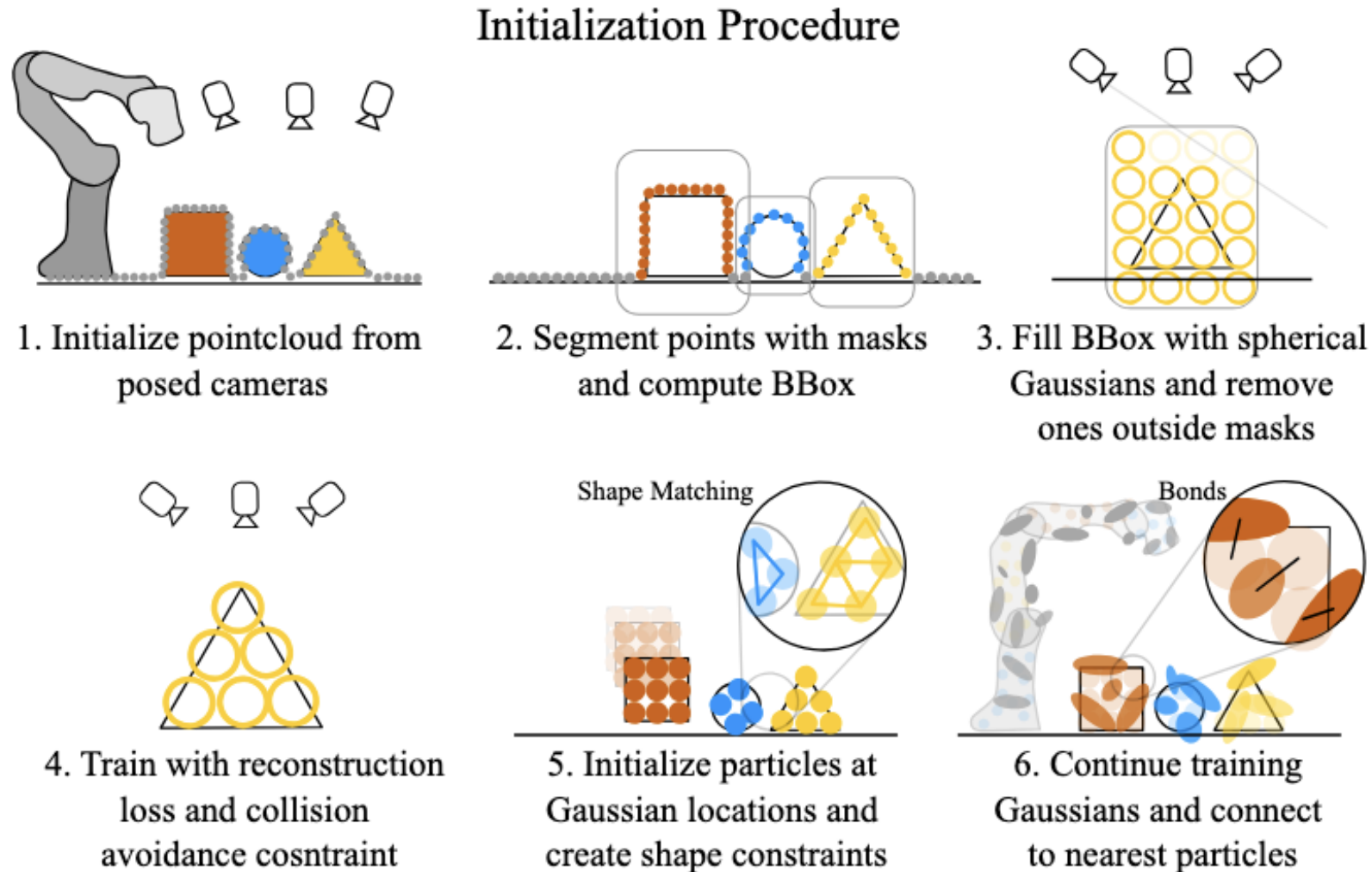
Problem: How to Couple 3D Gaussian Splatting with Particle Presentations of PBD System?



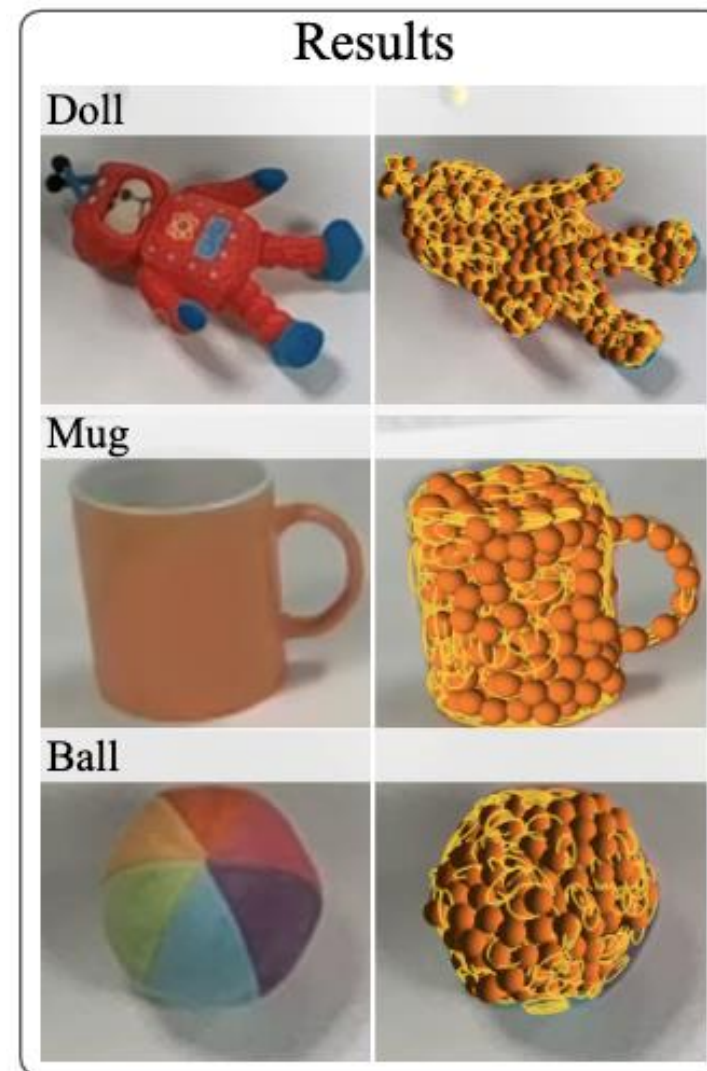
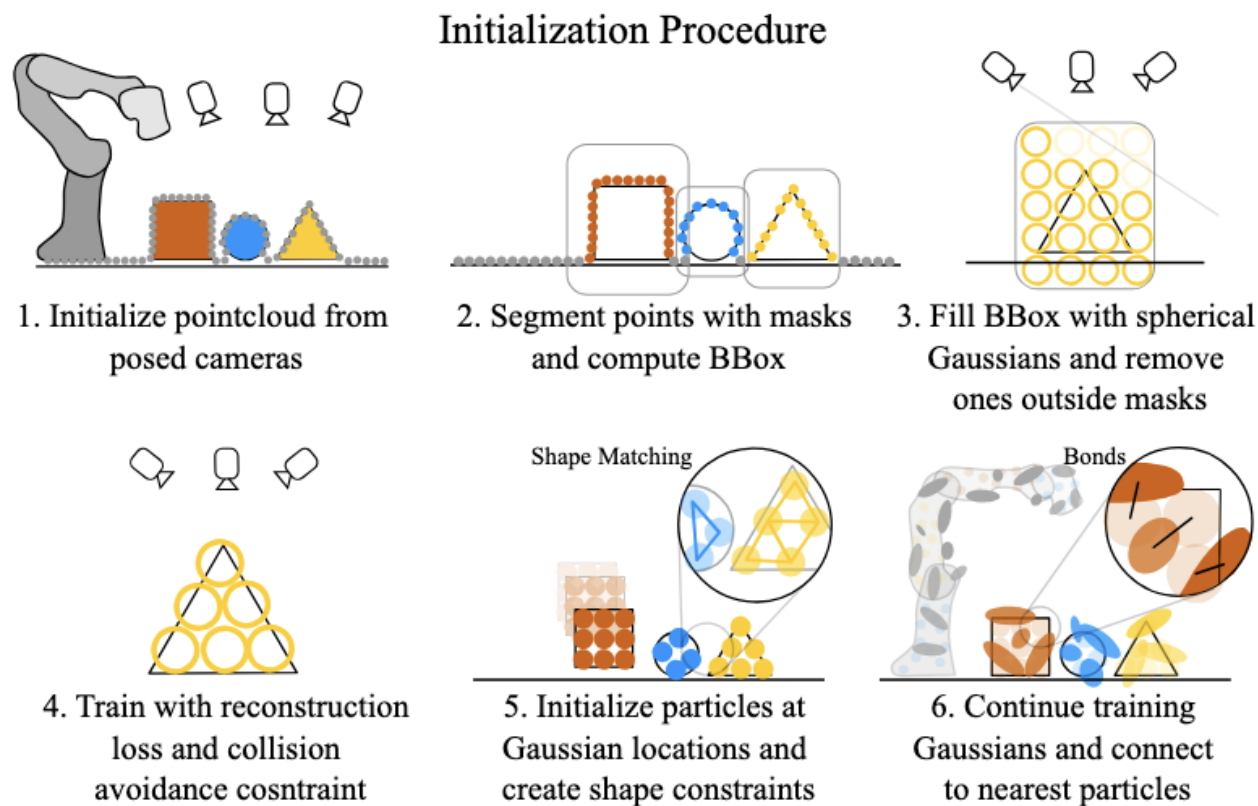
Problem: How to Couple 3D Gaussian Splatting with Particle Presentations of PBD System?



Problem: How to Couple 3D Gaussian Splatting with Particle Presentations of PBD System?



Problem: How to Couple 3D Gaussian Splatting with Particle Presentations of PBD System?



PBD Constraints

- PBD acts on oriented particles. Each particle i has position $\mathbf{p}_i \in \mathbb{R}^3$, velocity $\mathbf{v}_i \in \mathbb{R}^3$, orientation $\mathbf{q}_i \in \mathbb{S}^3$, angular velocity $\boldsymbol{\omega}_i \in \mathbb{R}^3$, external force $\mathbf{f}_i \in \mathbb{R}^3$, radius $r_i \in \mathbb{R}^+$, and mass $m_i \in \mathbb{R}^+$. A particle may belong to a shape $\mathcal{S}_i$ and thus has its resting position $\bar{\mathbf{p}}_i$.
- Ground constraint: Prevent particles from penetrating the ground plane given by $(\mathbf{n}, d)$

$$\Delta \mathbf{p}_i^{\text{ground}} = C_{\text{ground}}(\mathbf{p}_i; \mathbf{n}, d) \cdot \mathbf{n}, \quad C_{\text{ground}}(\mathbf{p}_i; \mathbf{n}, d) = \min(\mathbf{n}^T \mathbf{p}_i + d - r_i, 0)$$

PBD Constraints

- PBD acts on oriented particles. Each particle i has position $\mathbf{p}_i \in \mathbb{R}^3$, velocity $\mathbf{v}_i \in \mathbb{R}^3$, orientation $\mathbf{q}_i \in \mathbb{S}^3$, angular velocity $\boldsymbol{\omega}_i \in \mathbb{R}^3$, external force $\mathbf{f}_i \in \mathbb{R}^3$, radius $r_i \in \mathbb{R}^+$, and mass $m_i \in \mathbb{R}^+$. A particle may belong to a shape $\mathcal{S}_i$ and thus has its resting position $\bar{\mathbf{p}}_i$.
- Particle collision constraint: Operate on a pair of particles i and j is used to model collisions

$$\Delta \mathbf{p}_i^{\text{col}} = \frac{w_i}{w_i + w_j} \frac{\mathbf{p}_i - \mathbf{p}_j}{\|\mathbf{p}_i - \mathbf{p}_j\|} C_{\text{col}}(\mathbf{p}_i, \mathbf{p}_j), \quad C_{\text{col}}(\mathbf{p}_i, \mathbf{p}_j) = \min(\|\mathbf{p}_i - \mathbf{p}_j\| - r_i - r_j, 0)$$

PBD Constraints

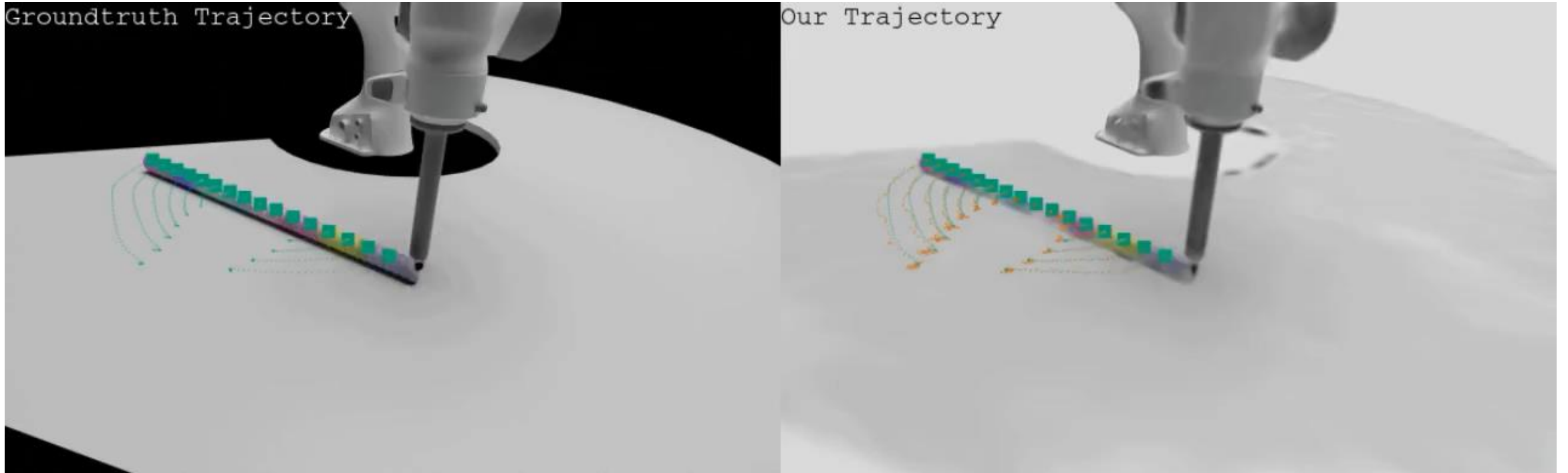
- PBD acts on oriented particles. Each particle i has position $\mathbf{p}_i \in \mathbb{R}^3$, velocity $\mathbf{v}_i \in \mathbb{R}^3$, orientation $\mathbf{q}_i \in \mathbb{S}^3$, angular velocity $\boldsymbol{\omega}_i \in \mathbb{R}^3$, external force $\mathbf{f}_i \in \mathbb{R}^3$, radius $r_i \in \mathbb{R}^+$, and mass $m_i \in \mathbb{R}^+$. A particle may belong to a shape S_i and thus has its resting position $\bar{\mathbf{p}}_i$.
- Shape matching constraint: Ensure a group of particles belonging to a particular object (either deformable or rigid) maintain their structure throughout the simulation

$$\boxed{\mathbf{A}_S} = \sum_{i \in S} \frac{1}{5} m_i \mathbf{R}_i + \mathbf{p}_i \bar{\mathbf{p}}_i^T - M \mathbf{c}_S \bar{\mathbf{c}}_S^T, \quad \mathbf{c}_S = \frac{\sum_{i \in S} m_i \mathbf{p}_i}{M}, \quad \bar{\mathbf{c}}_S = \frac{\sum_{i \in S} m_i \bar{\mathbf{p}}_i}{M}, \quad M = \sum_{i \in S} m_i$$

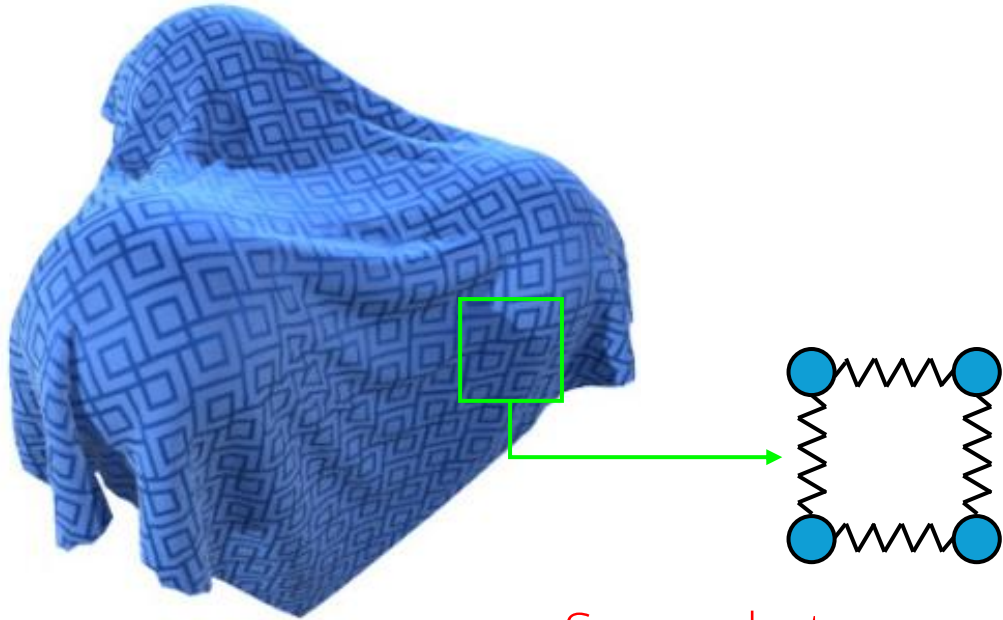
$\mathbf{A}_S = \mathbf{R}_S \mathbf{S}$ can be decomposed into a rotation matrix $\mathbf{R}_S$ and a symmetrical matrix $\mathbf{S}$

$$\Delta \mathbf{p}_i^{\text{shape}} = k_S [\mathbf{R}_S (\bar{\mathbf{p}}_i - \bar{\mathbf{c}}_S) + \mathbf{c} - \mathbf{p}_i]$$

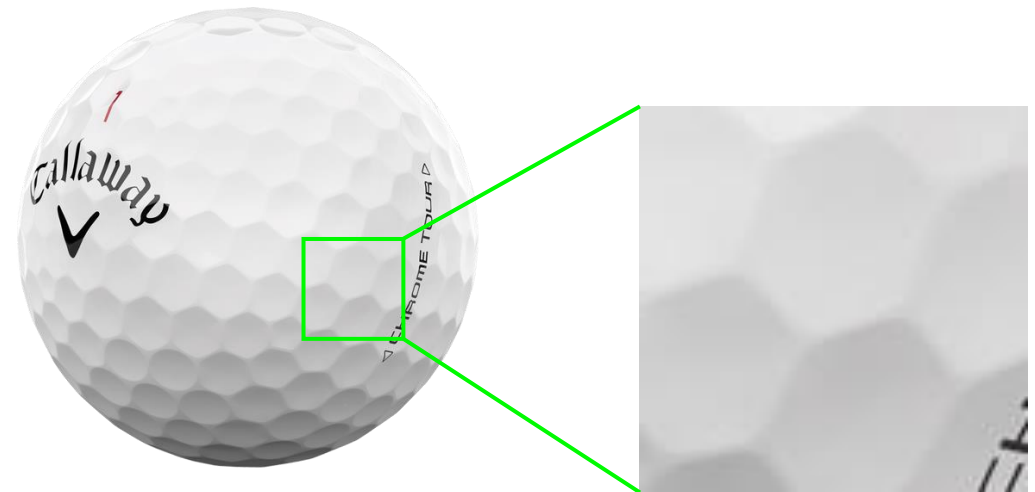
Results: Real-to-Sim Demo



Mass-Spring System and Particle-based PBD Do Not Model Continuum Mechanics

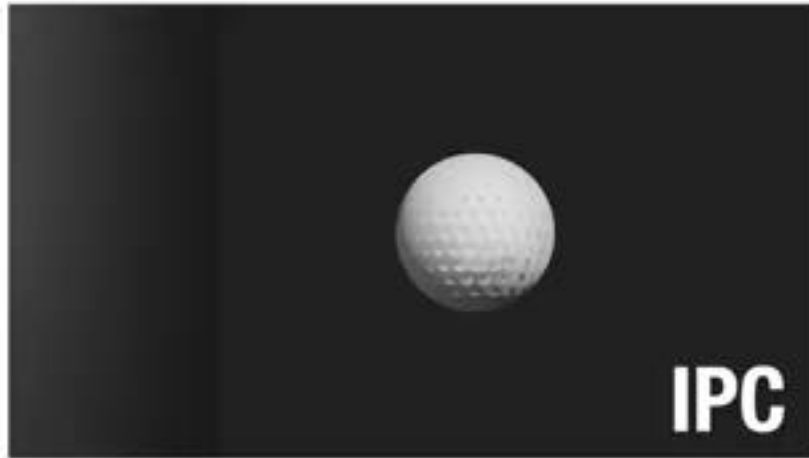


Spaces between nodes are empty: no mass and energy

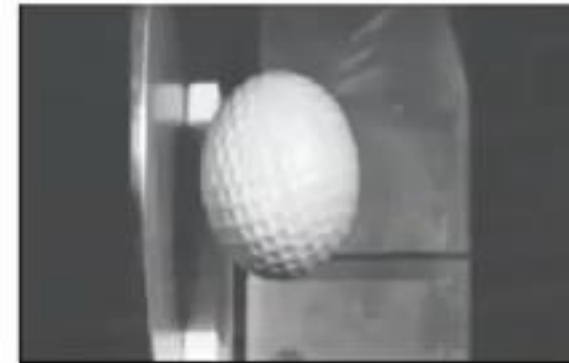


Spaces are continuously filled

Common Objects are Elastic, Even Though They Look Like Rigid Bodies



5e-4X



**Recorded
Footage**



IPC Velocity Magnitude

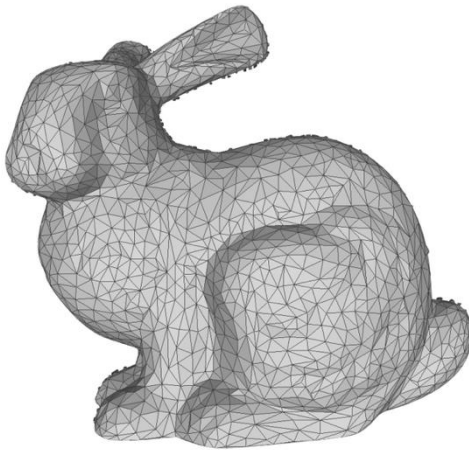
Continuum Mechanics

- How did every point in the object change shape?
- How to represent a continuum object?

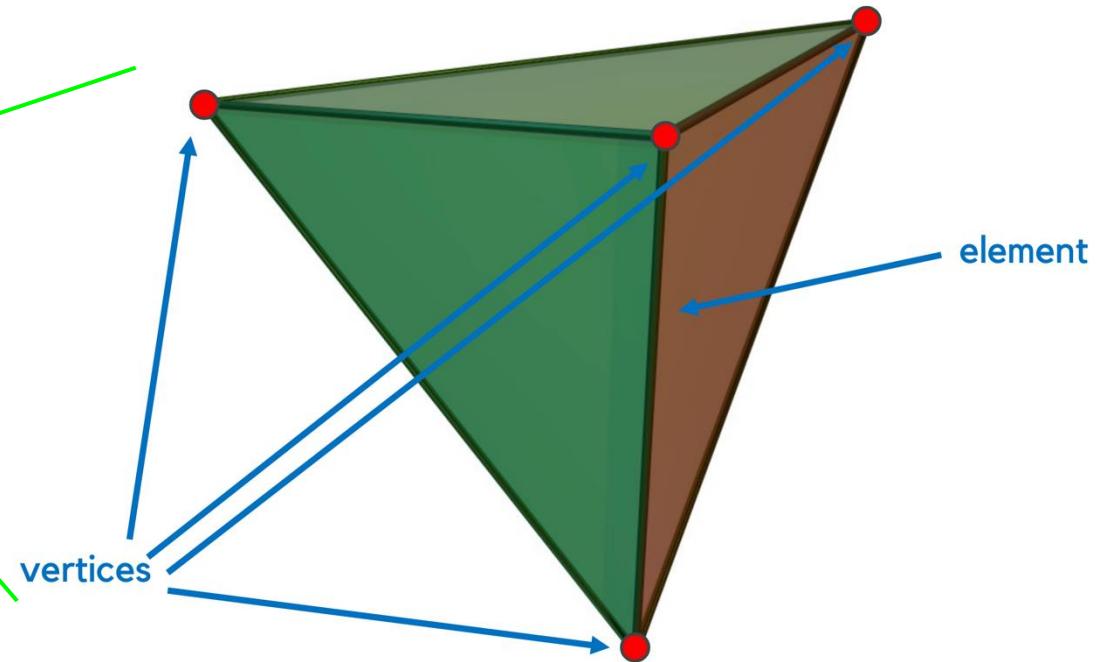
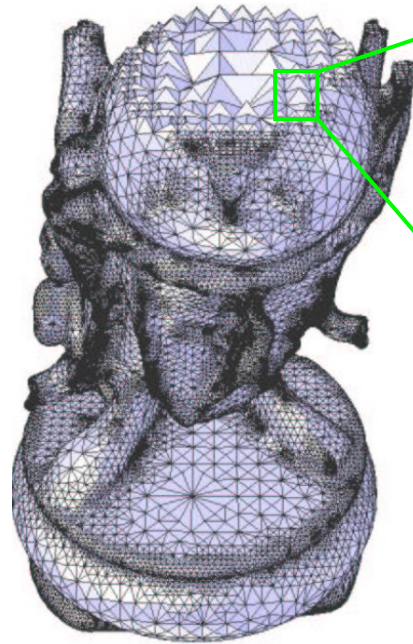


Tetrahedral Finite Elements

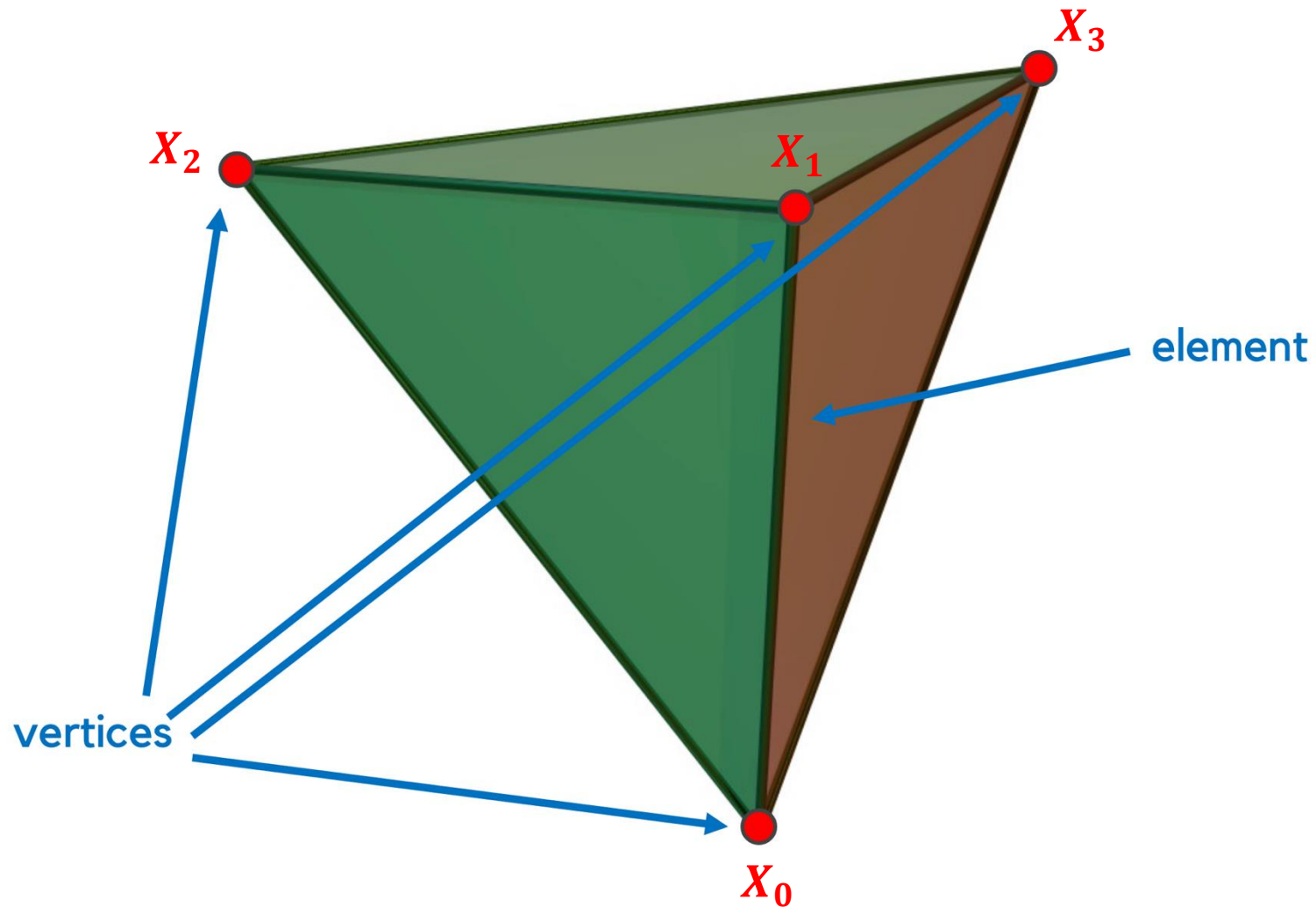
Mesh



Tetrahedral Mesh



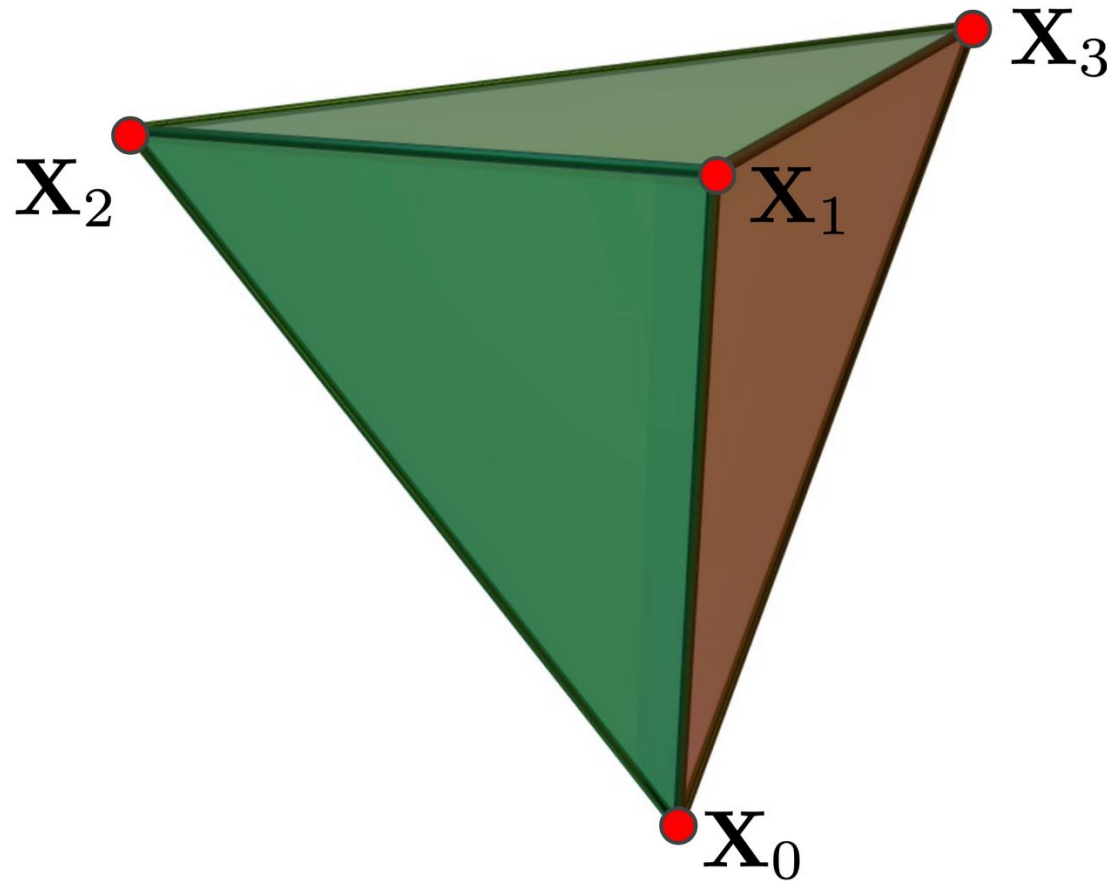
Tetrahedral Finite Elements



$$\mathbf{q} = \begin{pmatrix} X_0 \\ X_1 \\ X_2 \\ X_3 \end{pmatrix}$$

$$\dot{\mathbf{q}} = \begin{pmatrix} \dot{X}_0 \\ \dot{X}_1 \\ \dot{X}_2 \\ \dot{X}_3 \end{pmatrix}$$

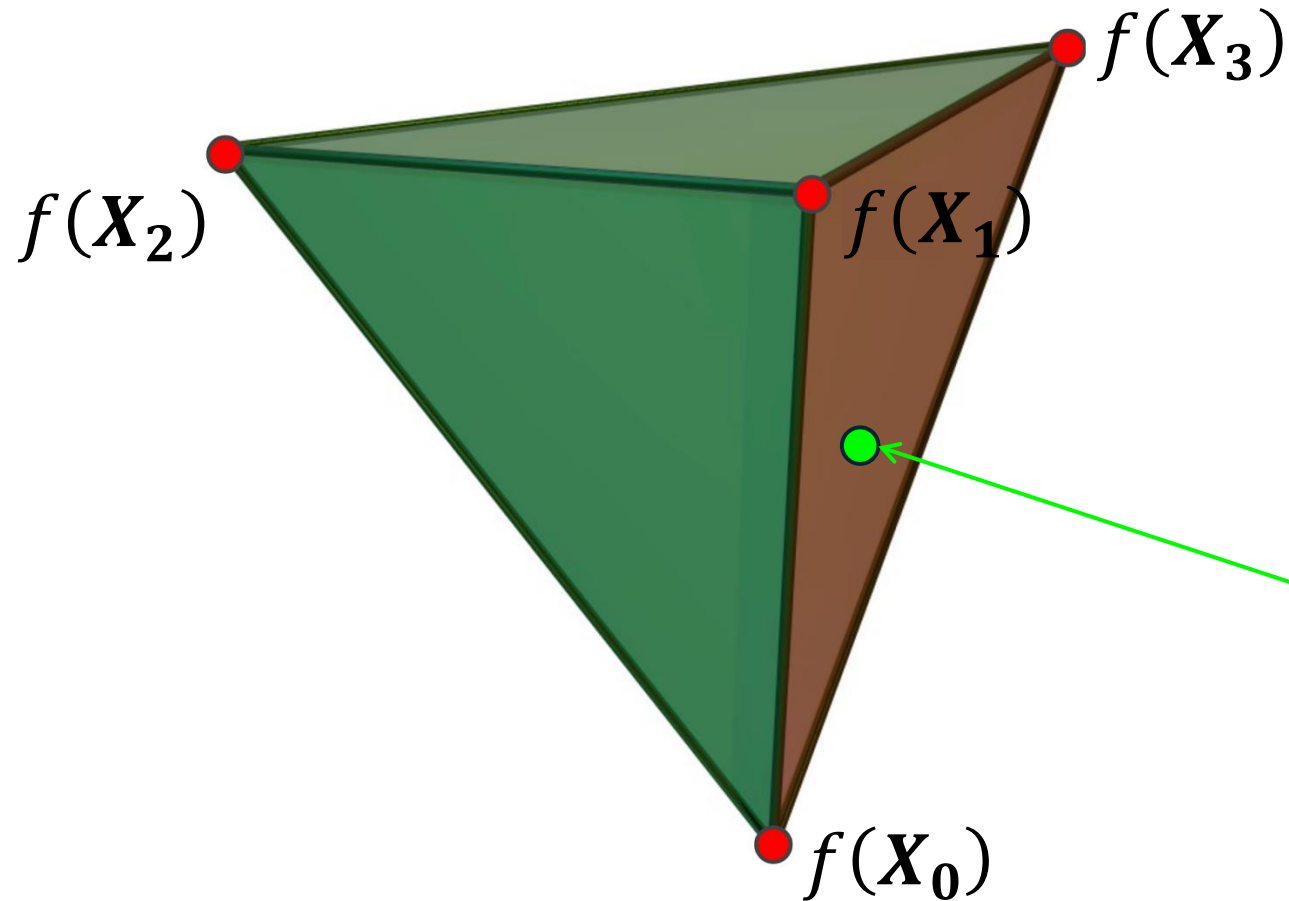
Generalized Coordinates for Tetrahedra Finite Elements



- $\mathbf{X}$ can be a set of parameters that specify the configuration of a physical system at each point in time (e.g. rotational angle in a pendulum system)

$$\mathbf{q} = \begin{pmatrix} \mathbf{X}_0 \\ \mathbf{X}_1 \\ \mathbf{X}_2 \\ \mathbf{X}_3 \end{pmatrix} \quad \dot{\mathbf{q}} = \begin{pmatrix} \dot{\mathbf{X}}_0 \\ \dot{\mathbf{X}}_1 \\ \dot{\mathbf{X}}_2 \\ \dot{\mathbf{X}}_3 \end{pmatrix}$$

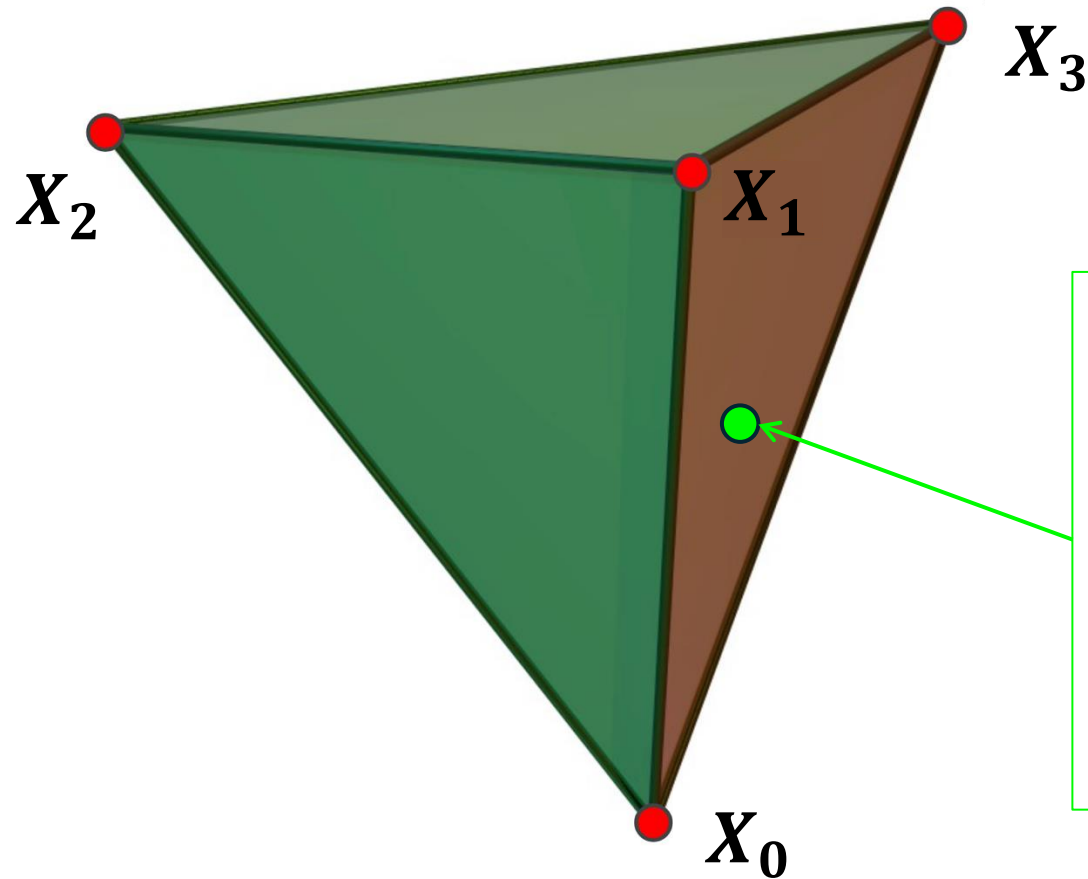
Why Do We Need Such a Representation?



- The value of any point in a tetrahedra can be linearly interpolated
- We don't need to keep track of the deformation of every point but few vertices

$$f(\mathbf{X}) = \sum_{i=0}^3 f(\mathbf{X}_i) \phi_i(\mathbf{X})$$

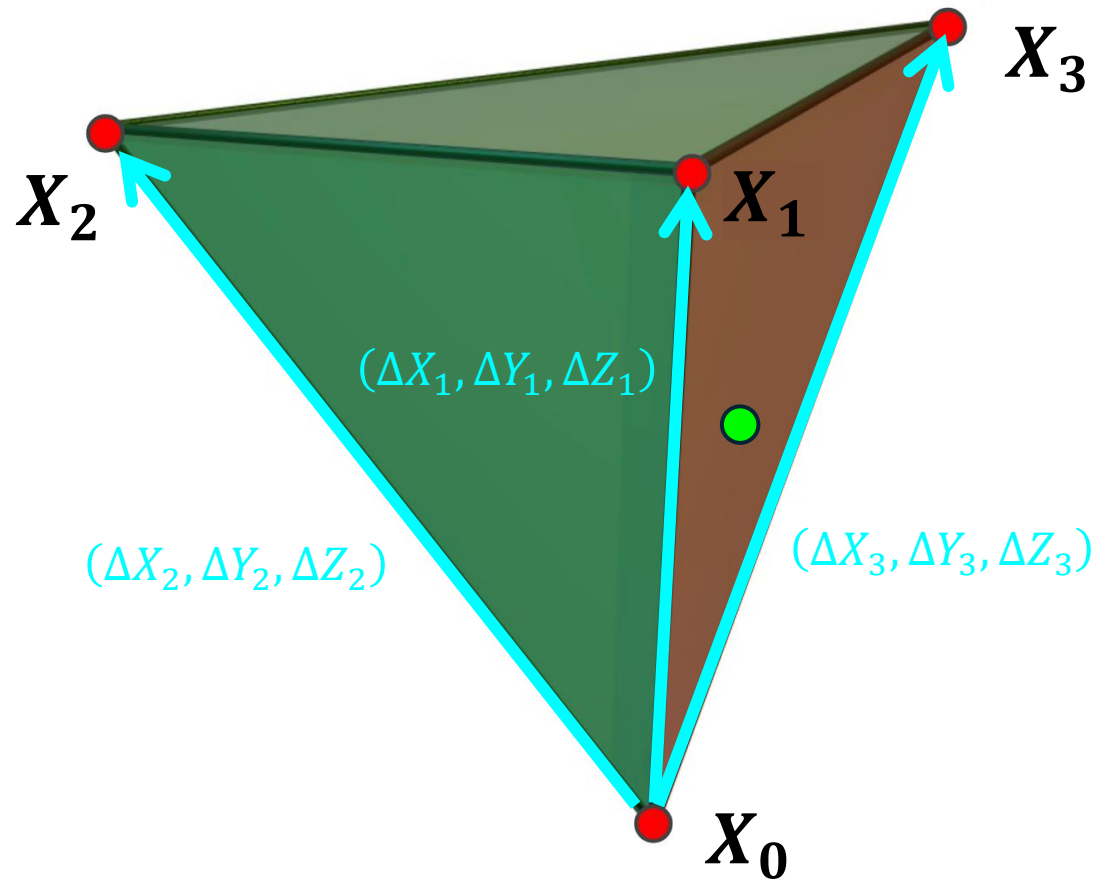
Why Do We Need Such a Representation?



- The value of any point in a tetrahedra can be linearly interpolated

$$\mathbf{X} = \sum_{i=0}^3 \mathbf{X}_i \phi_i(\mathbf{X})$$
$$\Rightarrow \begin{pmatrix} X \\ Y \\ Z \end{pmatrix} = \begin{pmatrix} X_0 & X_1 & X_2 & X_3 \\ Y_0 & Y_1 & Y_2 & Y_3 \\ Z_0 & Z_1 & Z_2 & Z_3 \end{pmatrix} \begin{pmatrix} \phi_0(\mathbf{X}) \\ \phi_1(\mathbf{X}) \\ \phi_2(\mathbf{X}) \\ \phi_3(\mathbf{X}) \end{pmatrix}$$

Why Do We Need Such a Representation?

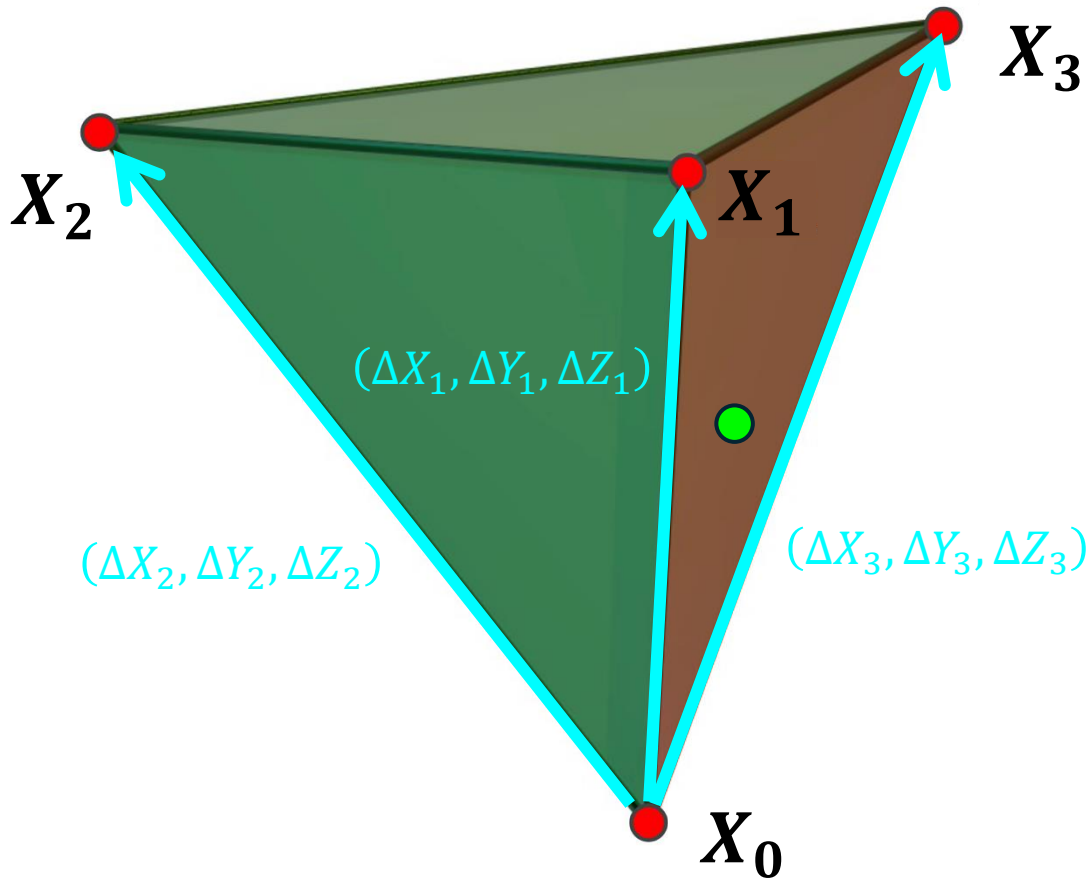


$$\begin{pmatrix} X \\ Y \\ Z \end{pmatrix} = \begin{pmatrix} X_0 & X_1 & X_2 & X_3 \\ Y_0 & Y_1 & Y_2 & Y_3 \\ Z_0 & Z_1 & Z_2 & Z_3 \end{pmatrix} \begin{pmatrix} \phi_0(\mathbf{X}) \\ \phi_1(\mathbf{X}) \\ \phi_2(\mathbf{X}) \\ \phi_3(\mathbf{X}) \end{pmatrix}$$

$$\text{Let } \phi_0(\mathbf{X}) = 1 - \phi_1(\mathbf{X}) - \phi_2(\mathbf{X}) - \phi_3(\mathbf{X})$$

$$\Rightarrow \begin{pmatrix} X - X_0 \\ Y - Y_0 \\ Z - Z_0 \end{pmatrix} = \begin{pmatrix} \Delta X_1 & \Delta X_2 & \Delta X_3 \\ \Delta Y_1 & \Delta Y_2 & \Delta Y_3 \\ \Delta Z_1 & \Delta Z_2 & \Delta Z_3 \end{pmatrix} \begin{pmatrix} \phi_1(\mathbf{X}) \\ \phi_2(\mathbf{X}) \\ \phi_3(\mathbf{X}) \end{pmatrix}$$

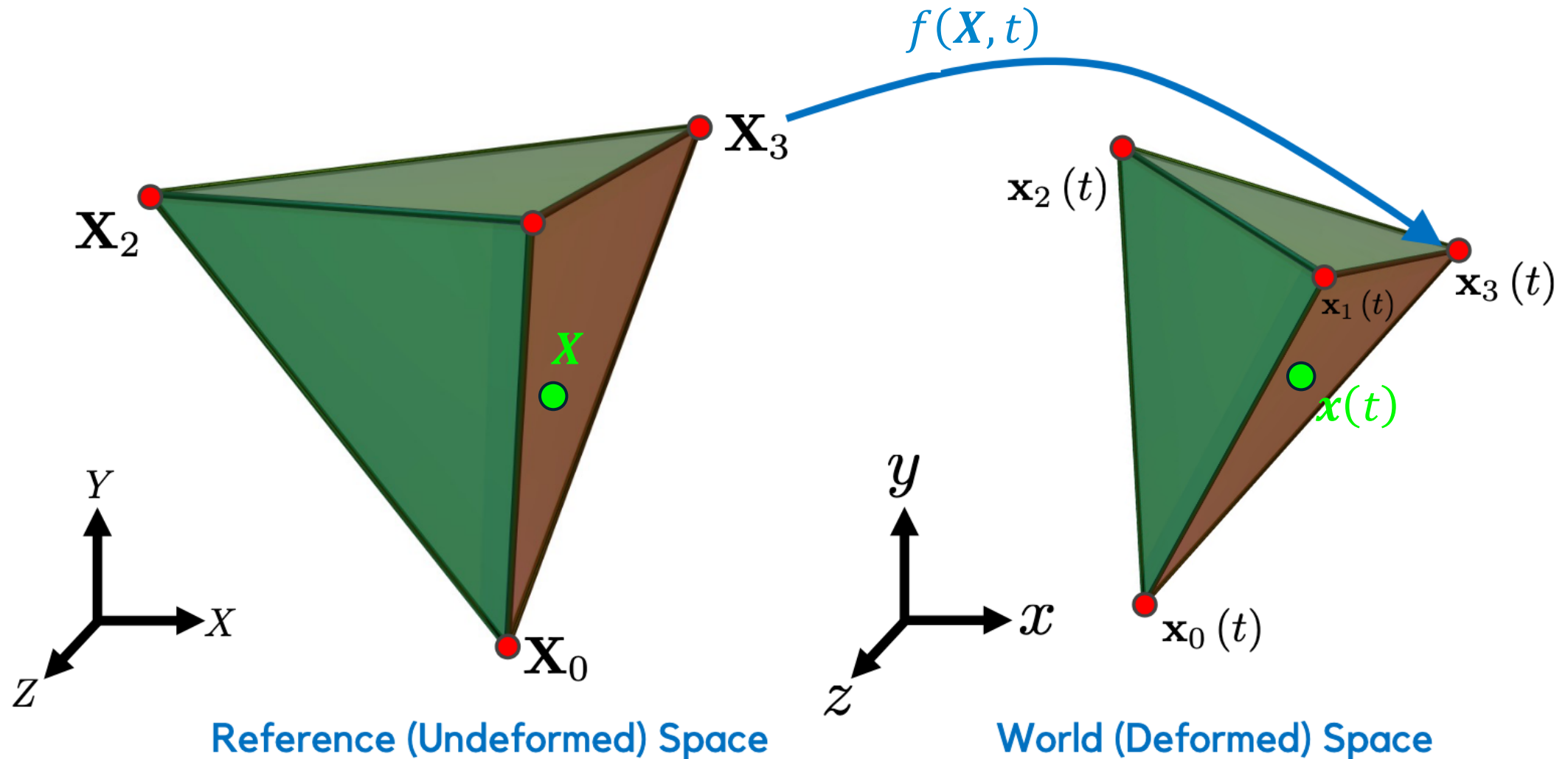
Barycentric Coordinates



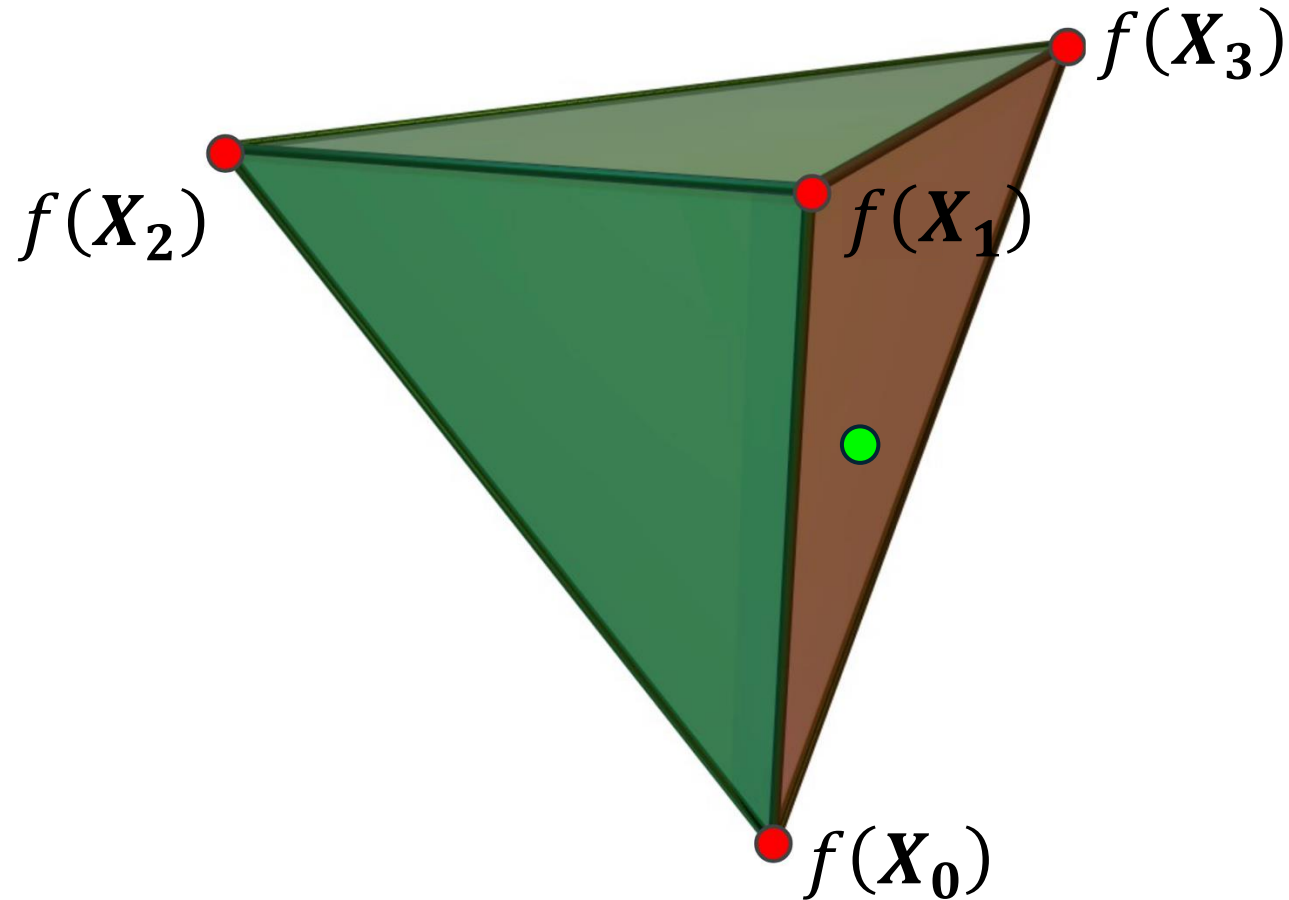
$$\begin{pmatrix} X - X_0 \\ Y - Y_0 \\ Z - Z_0 \end{pmatrix} = \begin{pmatrix} \Delta X_1 & \Delta X_2 & \Delta X_3 \\ \Delta Y_1 & \Delta Y_2 & \Delta Y_3 \\ \Delta Z_1 & \Delta Z_2 & \Delta Z_3 \end{pmatrix} \begin{pmatrix} \phi_1(\mathbf{X}) \\ \phi_2(\mathbf{X}) \\ \phi_3(\mathbf{X}) \end{pmatrix}$$
$$\Rightarrow \begin{pmatrix} \phi_1(\mathbf{X}) \\ \phi_2(\mathbf{X}) \\ \phi_3(\mathbf{X}) \end{pmatrix} = \begin{pmatrix} \Delta X_1 & \Delta X_2 & \Delta X_3 \\ \Delta Y_1 & \Delta Y_2 & \Delta Y_3 \\ \Delta Z_1 & \Delta Z_2 & \Delta Z_3 \end{pmatrix}^{-1} \begin{pmatrix} X - X_0 \\ Y - Y_0 \\ Z - Z_0 \end{pmatrix}$$

We can infer the interpolation weights from an undeformed tetrahedron

Material Deformation



Material Deformation

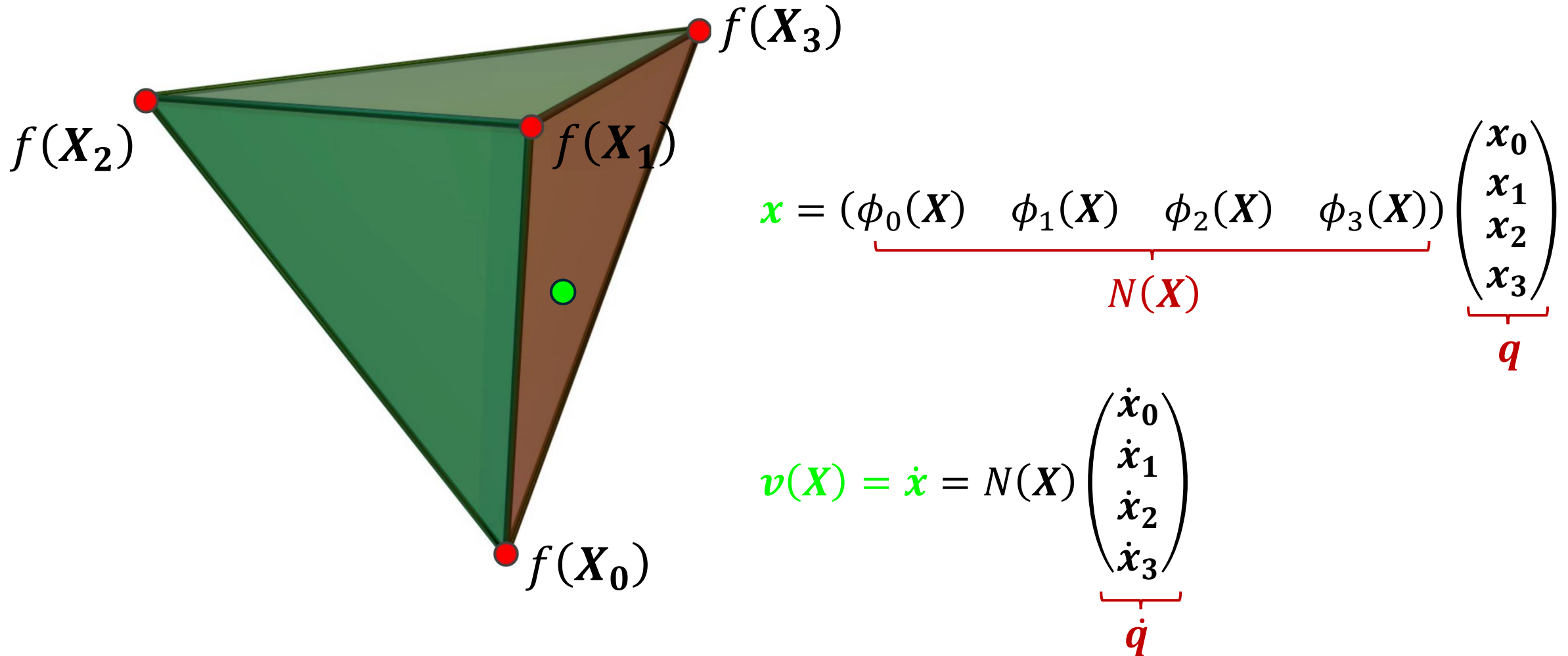


- The value of any point in a tetrahedra can be linearly interpolated
- We don't need to keep track of the deformation of every point but few vertices

$$\mathbf{x}(t) = \sum_{i=0}^3 \mathbf{x}_i(t) \phi_i(\mathbf{X})$$

Deformed vertex
position

Material Deformation



In the Next Lecture, We'll Introduce a More General Framework to Derive Equations of Motion

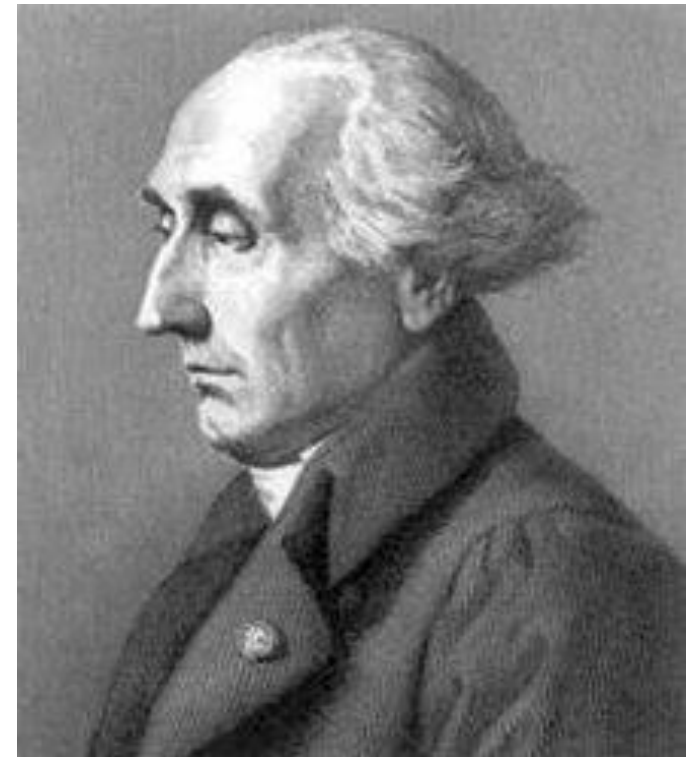
- The Lagrangian:

$$L = T - V$$

Kinetic energy Potential energy

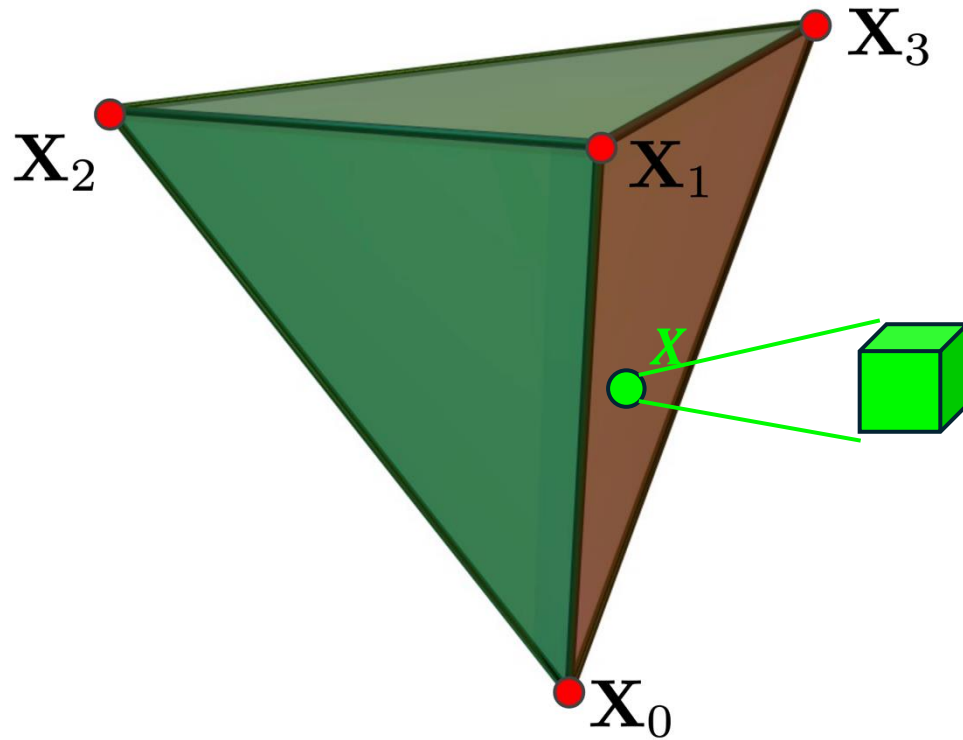
- Euler-Lagrange Equation:

$$\frac{d}{dt} \frac{\partial L}{\partial \dot{\mathbf{q}}} = - \frac{\partial L}{\partial \mathbf{q}}$$



Joseph-Louis Lagrange

Kinetic Energy of a Deformed Material



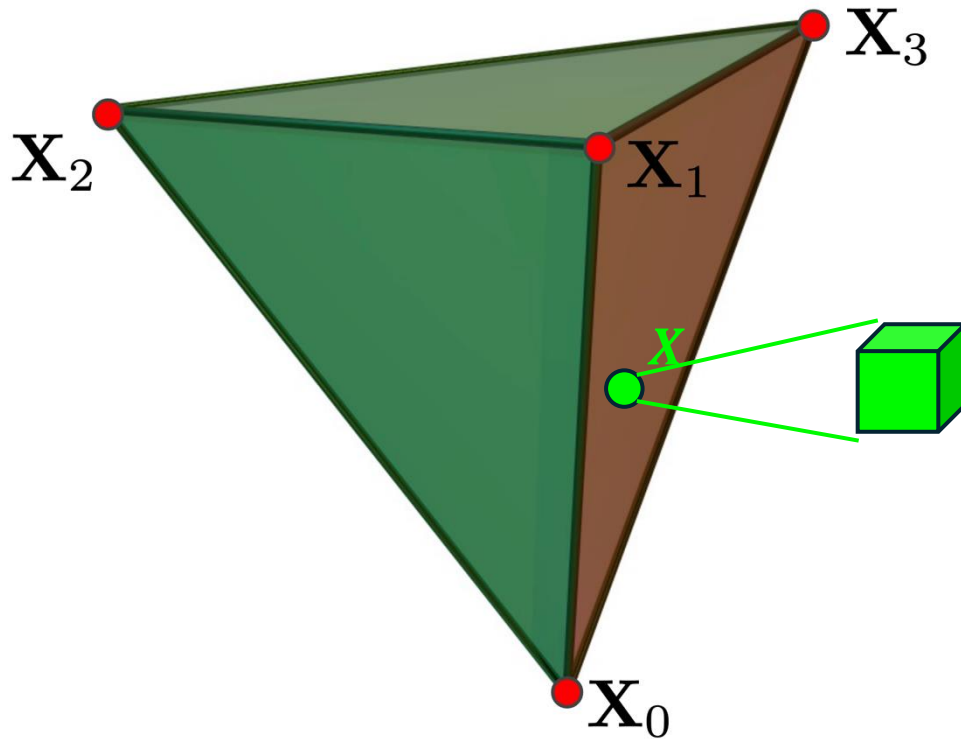
- Kinetic energy at $\mathbf{X}$:

$$\frac{1}{2} \underset{\substack{\text{unit density}}}{\rho} \|v(\mathbf{X})\|_2^2 \overset{\substack{\text{infinitesimal volume}}}{d\Omega}$$

- Kinetic energy of the tetrahedral

$$\begin{aligned} & \int_{\Omega} \frac{1}{2} \rho \|v(\mathbf{X})\|_2^2 d\Omega \\ & \quad \Omega \rightarrow \text{Tetrahedra domain} \\ & \Rightarrow \frac{1}{2} \int_{\Omega} \rho (N(\mathbf{X}) \dot{\mathbf{q}})^T N(\mathbf{X}) \dot{\mathbf{q}} d\Omega \\ & \Rightarrow \frac{1}{2} \dot{\mathbf{q}}^T \left(\int_{\Omega} \rho N(\mathbf{X})^T N(\mathbf{X}) d\Omega \right) \dot{\mathbf{q}} \end{aligned}$$

Kinetic Energy of a Deformed Material



- Kinetic energy of the tetrahedral

$$\frac{1}{2} \dot{\mathbf{q}}^\top \left(\int_{\Omega} \rho \mathbf{N}(\mathbf{X})^\top \mathbf{N}(\mathbf{X}) d\Omega \right) \dot{\mathbf{q}}$$

↓

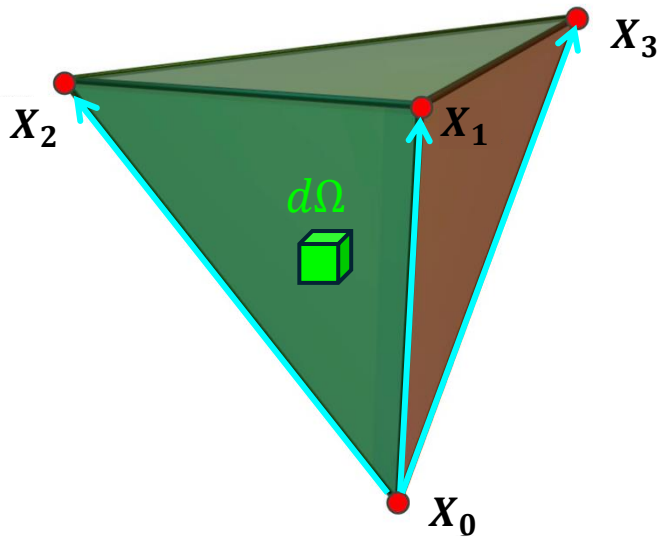
$$\int_{\Omega} \rho \begin{pmatrix} \phi_0 \phi_0 I & \phi_0 \phi_1 I & \phi_0 \phi_2 I & \phi_0 \phi_3 I \\ \phi_1 \phi_0 I & \phi_1 \phi_1 I & \phi_1 \phi_2 I & \phi_1 \phi_3 I \\ \phi_2 \phi_0 I & \phi_2 \phi_1 I & \phi_2 \phi_2 I & \phi_2 \phi_3 I \\ \phi_3 \phi_0 I & \phi_3 \phi_1 I & \phi_3 \phi_2 I & \phi_3 \phi_3 I \end{pmatrix} d\Omega$$

- Evaluate at each term separately:

$$\int_{\Omega} \rho \phi_r \phi_s I d\Omega$$

The Differential Volume Element

- Integral over the tetrahedral domain Ω : $\int_{\Omega} \rho \phi_r \phi_s I d\Omega$
- The integral is difficult to calculate using the Cartesian coordinates
- Idea: Transform the Cartesian coordinates into Barycentric coordinates

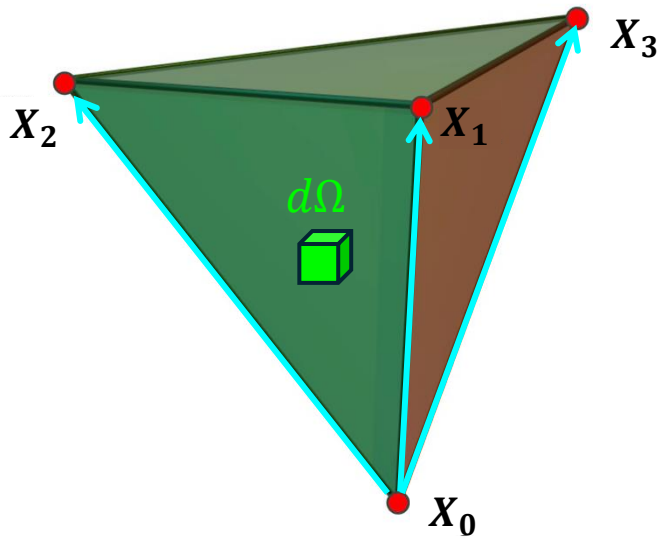


We have $\phi_0(\mathbf{X}) = 1 - \phi_1(\mathbf{X}) - \phi_2(\mathbf{X}) - \phi_3(\mathbf{X})$

$$\begin{pmatrix} X \\ Y \\ Z \end{pmatrix} = \underbrace{\begin{pmatrix} X_0 & X_1 & X_2 & X_3 \\ Y_0 & Y_1 & Y_2 & Y_3 \\ Z_0 & Z_1 & Z_2 & Z_3 \end{pmatrix}}_{\text{Cartesian coordinates}} \underbrace{\begin{pmatrix} \phi_0(\mathbf{X}) \\ \phi_1(\mathbf{X}) \\ \phi_2(\mathbf{X}) \\ \phi_3(\mathbf{X}) \end{pmatrix}}_{\text{Barycentric coordinates}}$$

The Differential Volume Element

- Integral over the tetrahedral domain Ω : $\int_{\Omega} \rho \phi_r \phi_s I d\Omega$
- The integral is difficult to calculate using the Cartesian coordinates
- Idea: Transform the Cartesian coordinates into Barycentric coordinates



$$\begin{pmatrix} X \\ Y \\ Z \end{pmatrix} = \begin{pmatrix} X_0 & (X_1 - X_0)\phi_1 & (X_2 - X_0)\phi_2 & (X_3 - X_0)\phi_3 \\ Y_0 & (Y_1 - Y_0)\phi_1 & (Y_2 - Y_0)\phi_2 & (Y_3 - Y_0)\phi_3 \\ Z_0 & (Z_1 - Z_0)\phi_1 & (Z_2 - Z_0)\phi_2 & (Z_3 - Z_0)\phi_3 \end{pmatrix}$$

Remember that We Transformed Two Coordinate System with Jacobian $J_{\mathbf{u}_k}$

- Projection is not affine mapping

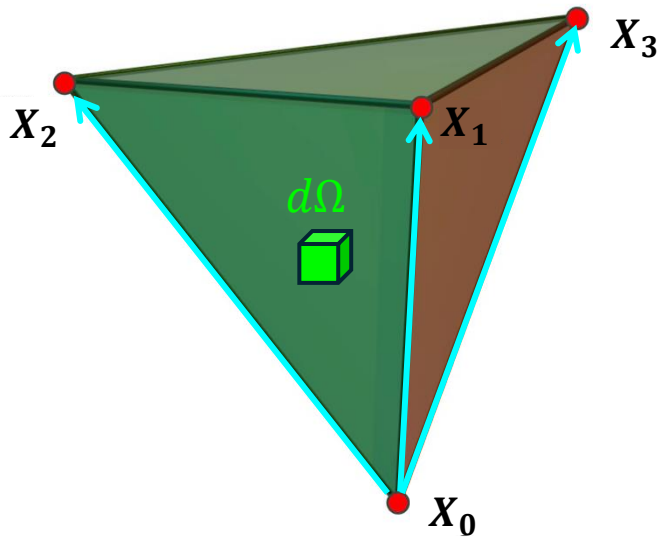
$$\begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix} = m(\mathbf{u}) = \begin{pmatrix} u_0/u_2 \\ u_1/u_2 \\ \|\mathbf{u}\| \end{pmatrix}$$

- We can approximate with 1st order Taylor Expansion at mean $\mathbf{u}_k$ and $\mathbf{x}_k$ of two spaces as:

$$m_{\mathbf{u}_k}(\mathbf{u}) = \mathbf{x}_k + J_{\mathbf{u}_k} \cdot (\mathbf{u} - \mathbf{u}_k), \quad \text{where } J_{\mathbf{u}_k} = \frac{\partial m}{\partial \mathbf{u}}(\mathbf{u}_k)$$

The Differential Volume Element

- Integral over the tetrahedral domain Ω : $\int_{\Omega} \rho \phi_r \phi_s I d\Omega$
- The integral is difficult to calculate using the Cartesian coordinates
- Idea: Transform the Cartesian coordinates into Barycentric coordinates



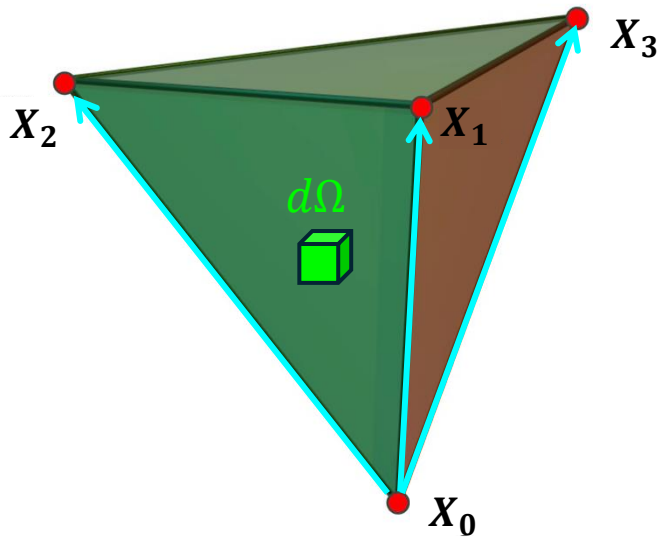
$$\begin{pmatrix} X \\ Y \\ Z \end{pmatrix} = \begin{pmatrix} X_0 & (X_1 - X_0)\phi_1 & (X_2 - X_0)\phi_2 & (X_3 - X_0)\phi_3 \\ Y_0 & (Y_1 - Y_0)\phi_1 & (Y_2 - Y_0)\phi_2 & (Y_3 - Y_0)\phi_3 \\ Z_0 & (Z_1 - Z_0)\phi_1 & (Z_2 - Z_0)\phi_2 & (Z_3 - Z_0)\phi_3 \end{pmatrix}$$

$$\Rightarrow \begin{pmatrix} X \\ Y \\ Z \end{pmatrix} = \begin{pmatrix} X_0 \\ Y_0 \\ Z_0 \end{pmatrix} + J \begin{pmatrix} \phi_1 \\ \phi_2 \\ \phi_3 \end{pmatrix}$$

$\phi_1 = \phi_2 = \phi_3 = 0$ evaluated at $\mathbf{X}_0$

The Differential Volume Element

- Integral over the tetrahedral domain Ω : $\int_{\Omega} \rho \phi_r \phi_s I d\Omega$
- The integral is difficult to calculate using the Cartesian coordinates
- Idea: Transform the Cartesian coordinates into Barycentric coordinates



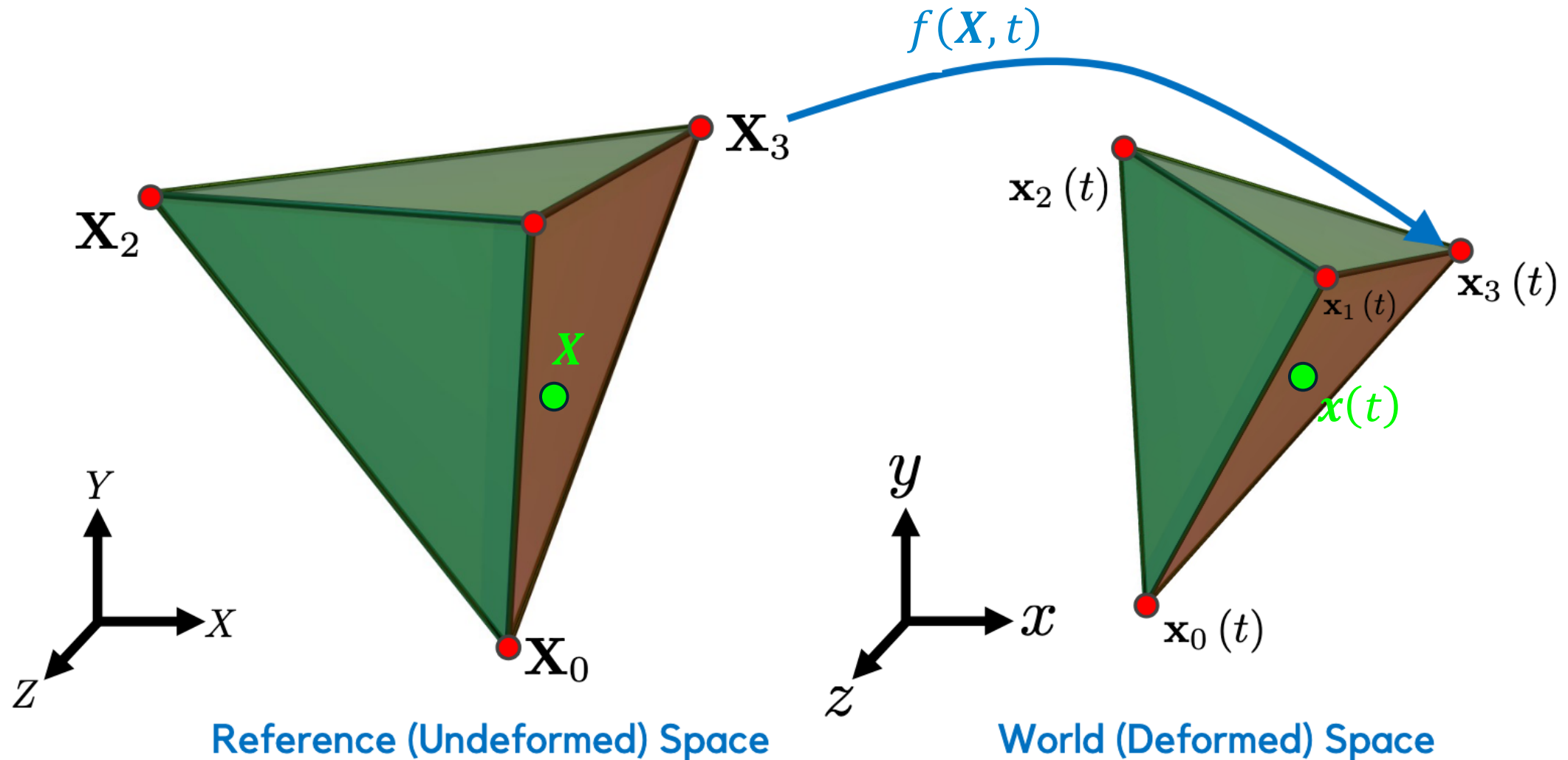
- Let V denotes the tetrahedron's volume, we have:

$$dV = |\det \mathbf{J}| d\phi_1 d\phi_2 d\phi_3 = 6V d\phi_1 d\phi_2 d\phi_3$$

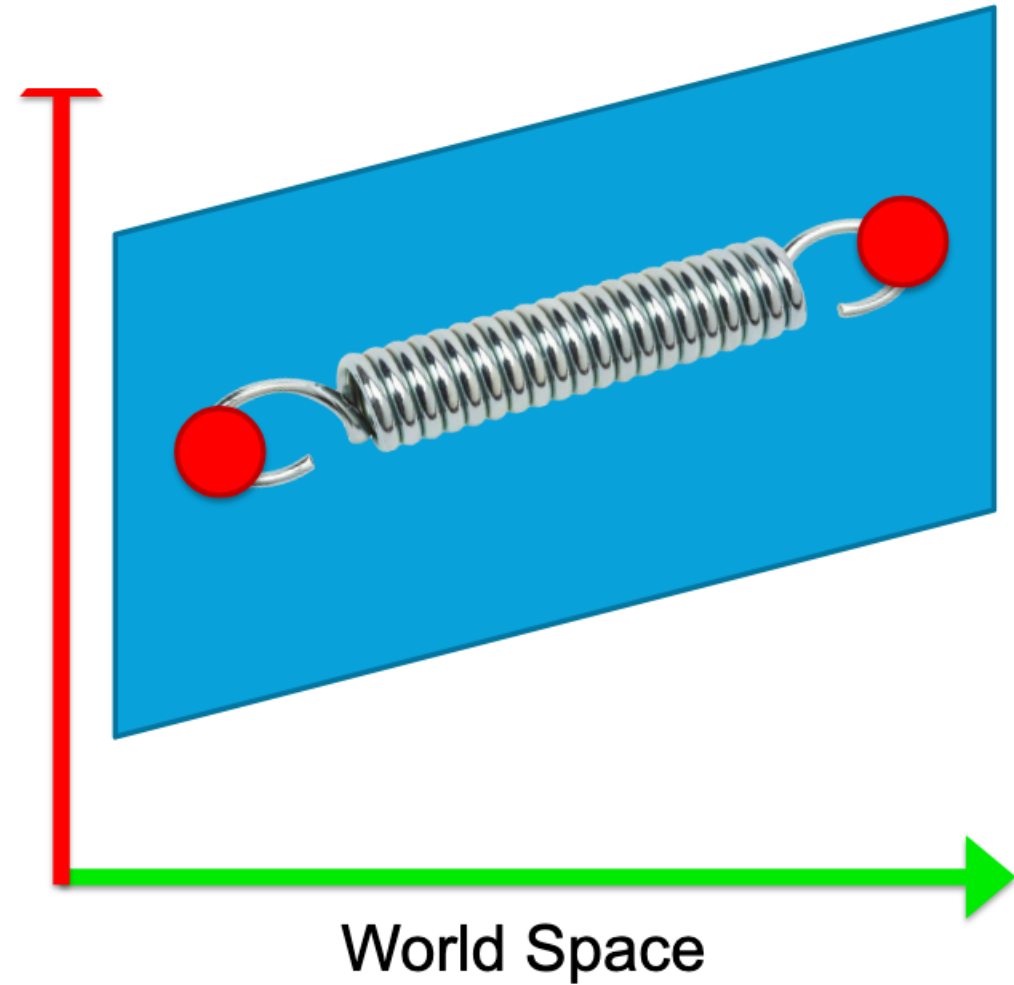
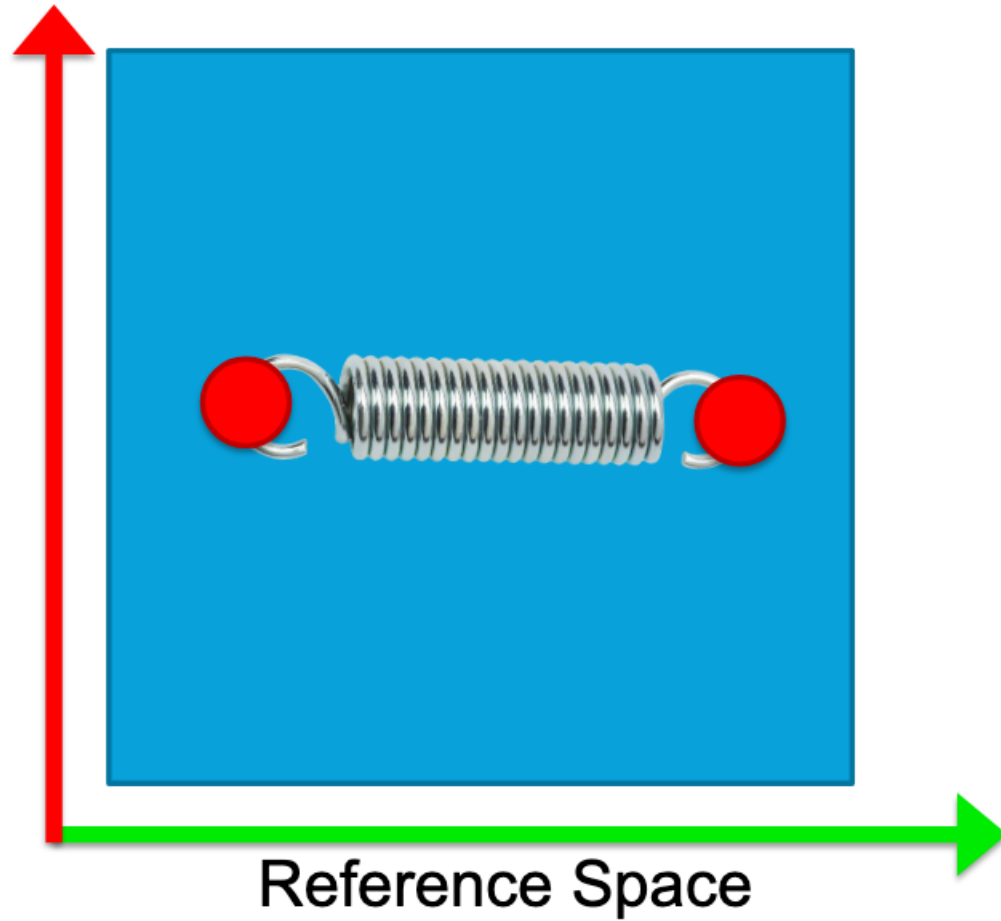
- We can calculate the integral:

$$\int_{\Omega} \rho \phi_r \phi_s I d\Omega = 6\rho V \int_0^1 \int_0^{1-\phi_1} \int_0^{1-\phi_1-\phi_2} (\phi_r \phi_s) d\phi_1 d\phi_2 d\phi_3$$

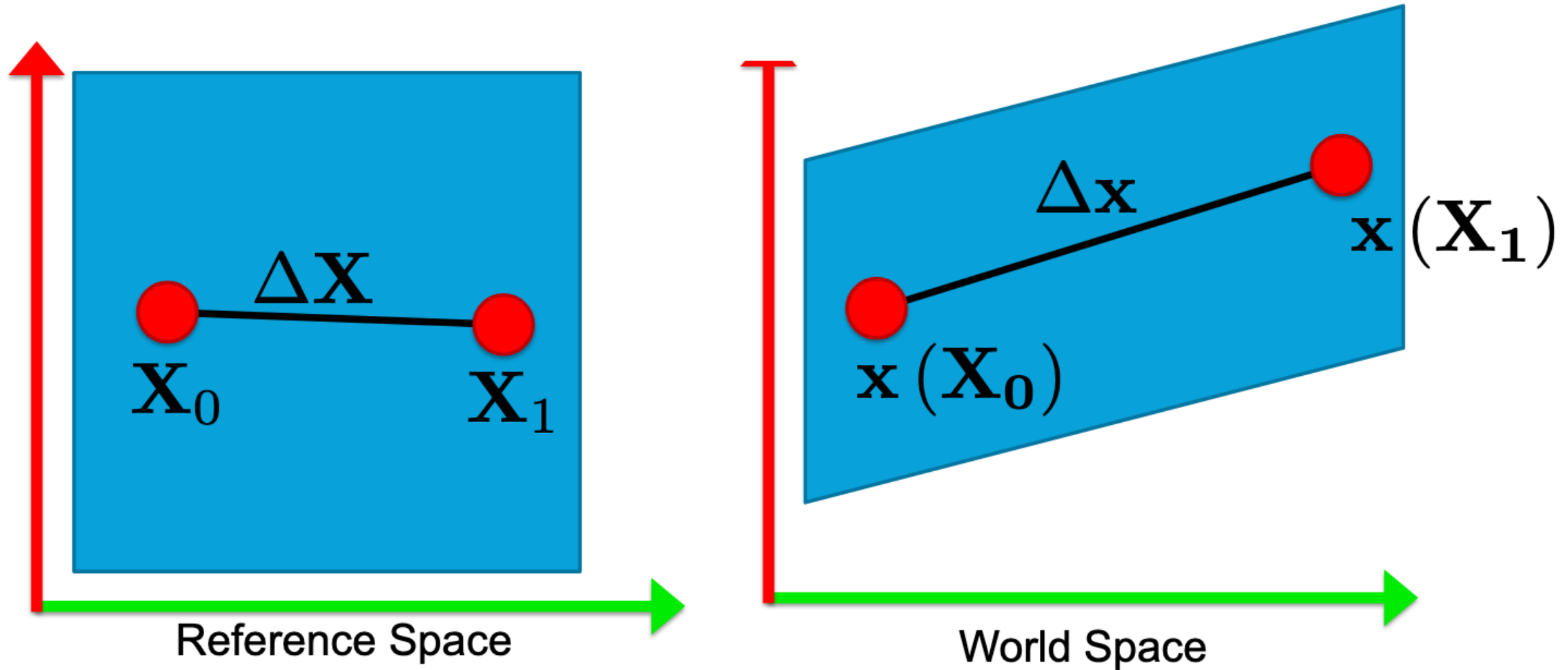
Potential Energy of Material Deformation



A Closer Look at Deformation

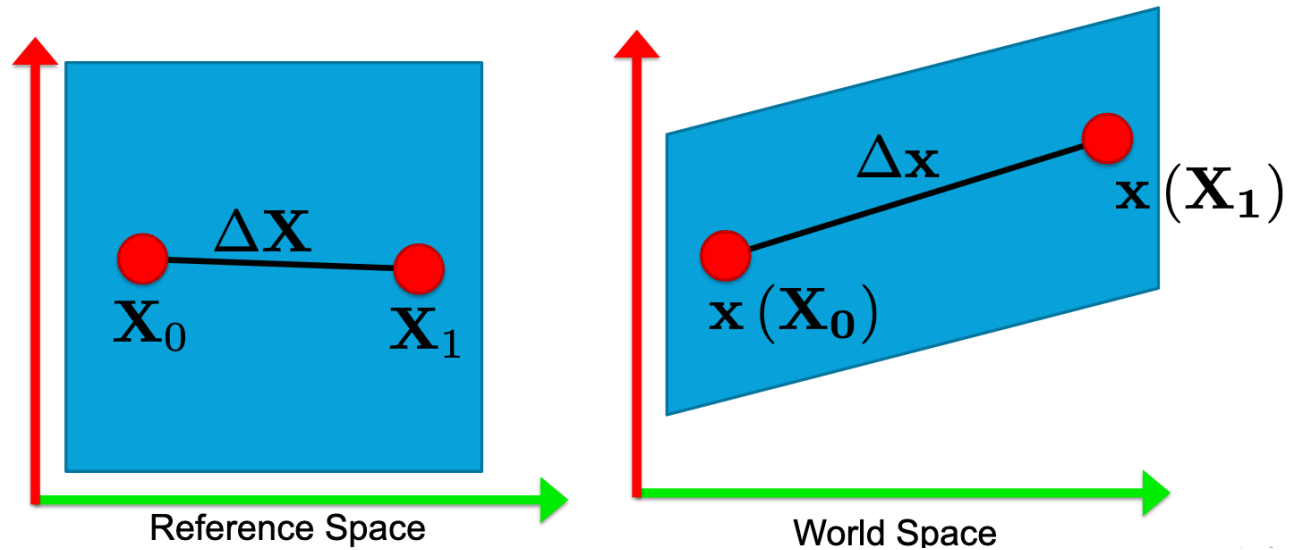


A Closer Look at Deformation



$$\Delta \mathbf{x} = \mathbf{x}(\mathbf{X}_1) - \mathbf{x}(\mathbf{X}_0)$$

A Closer Look at Deformation



$$\begin{aligned}
 \Delta \mathbf{x} &= f(\mathbf{X}_1) - f(\mathbf{X}_0) \\
 \Rightarrow \Delta \mathbf{x} &= f(\mathbf{X}_0 + \Delta \mathbf{X}) - f(\mathbf{X}_0) \\
 \Rightarrow \Delta \mathbf{x} &\approx \underbrace{f(\mathbf{X}_0) + \frac{\partial f}{\partial \mathbf{X}} \Delta \mathbf{X}}_{\text{Deformation gradient } F} - f(\mathbf{X}_0)
 \end{aligned}$$

Deformation gradient F

- Deformed length squared:

$$l^2 = \Delta \mathbf{x}^\top \Delta \mathbf{x}$$
- Rest length squared:

$$l_0^2 = \Delta \mathbf{X}^\top \Delta \mathbf{X}$$
- Strain:

$$\begin{aligned}
 l^2 - l_0^2 &= \Delta \mathbf{x}^\top \Delta \mathbf{x} - \Delta \mathbf{X}^\top \Delta \mathbf{X} \\
 &= \Delta \mathbf{X}^\top \mathbf{F}^\top \mathbf{F} \Delta \mathbf{X} - \Delta \mathbf{X}^\top \Delta \mathbf{X} \\
 &= \Delta \mathbf{X}^\top (\mathbf{F}^\top \mathbf{F} - \mathbf{I}) \Delta \mathbf{X}
 \end{aligned}$$
- Unit potential energy resulted from strain: $\Psi(\mathbf{F}(\mathbf{X}))$

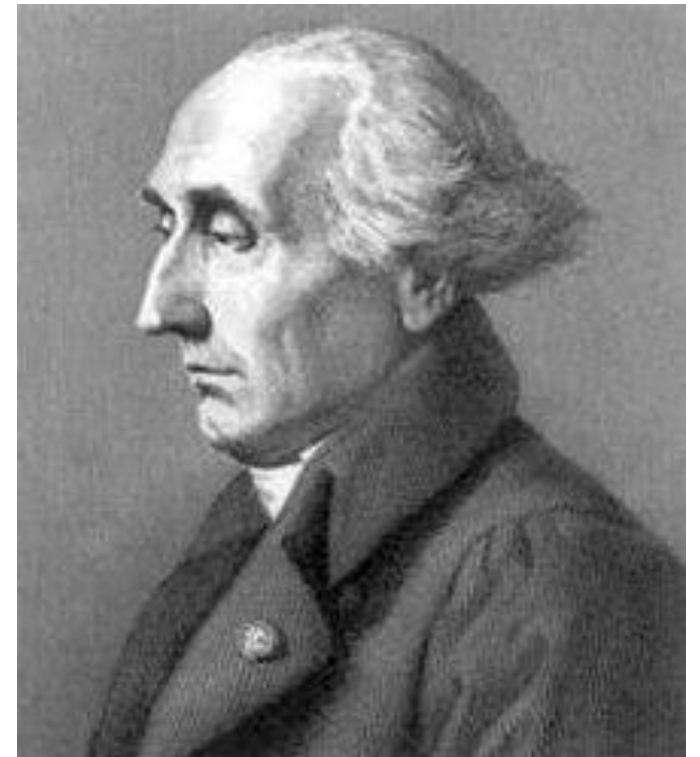
In the Next Lecture, We'll Introduce a More General Framework to Derive Equations of Motion

- The Lagrangian:

$$L = T - V$$

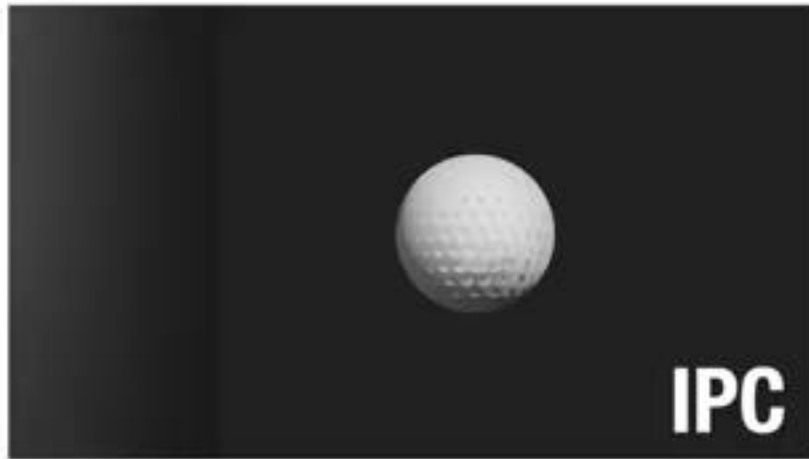
$$T = \frac{1}{2} \dot{\mathbf{q}}^T \left(\int_{\Omega} \rho \mathbf{N}(\mathbf{X})^T \mathbf{N}(\mathbf{X}) d\Omega \right) \dot{\mathbf{q}}$$

$$V = \sum_{j=0}^{m-1} vol_j \cdot \Psi(F_j(q_j))$$



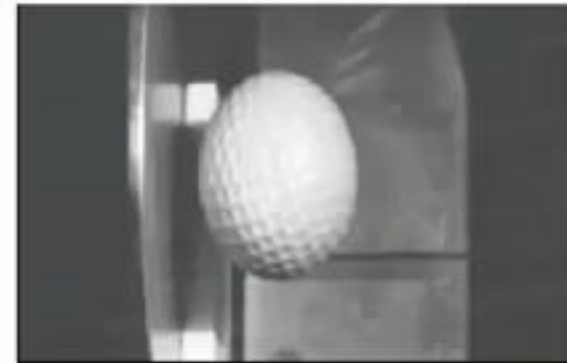
Joseph-Louis Lagrange

Why Do We Need Continuum Mechanics?




IPC Velocity Magnitude

5e-4X



**Recorded
Footage**

What We Will Cover the Next Week

- Physical modeling
 - Lagrangian Mechanics
 - Contact Modeling
 - Material Point Methods